title: "以下是 readme.md 的內容

根據新增的部分完善它

Deep Reinforcement Learning - HW1: Grid World P"

exported: 2026-03-18 03:03:04

url: https://gemini.google.com/app/00b7a24f82110e6f?hl=zh-TW

messages: 54

以下是 readme.md 的內容

根據新增的部分完善它

Deep Reinforcement Learning - HW1: Grid World P

You

作業內容 HW1-1: 網格地圖開發 目標：開發一個大小為 nxn 的網格地圖，允許用戶指定維度 'n'（範圍從 5 到9），使用 Flask 網頁應用程式。 功能要求： 用戶可以透過滑鼠點擊指定起始單元格（點擊後顯示為綠色）和結束單元格（點擊後顯示為紅色）。 用戶可以設定 n-2 個障礙物，透過滑鼠點擊將這些單元格變為灰色。 HW1-2: 策略顯示與價值評估 目標：顯示每個單元格的隨機生成行動（上下左右箭頭）作為策略。 功能要求：使用策略評估推導出每個狀態的價值 V(s)。

評分標準 HW1-1 網格地圖開發 60% 網格地圖功能完整性 30% 使用者界面友好性 15% 程式碼結構與可讀性 10% 網頁操作流暢度 5%

HW1-2 策略顯示與價值評估 40% 隨機生成行動顯示功能 20%。 策略評估的正確性 15% 程式碼結構與可讀性 5%

Gemini

這是一個非常經典的強化學習（Reinforcement Learning）基礎作業，結合了前端網頁互動與後端演算法。你的目標是建立一個「Grid World」環境，讓使用者設定初始狀態，然後在後端執行**策略評估（Policy Evaluation）**。

為了解決這個作業，我建議將任務拆解為「前端介面（HW1-1）」與「後端演算法（HW1-2）」兩個部分。以下是實作的架構指南與核心邏輯：

🛠️ 系統架構建議

1. **前端 (HTML / CSS / JavaScript)**：負責繪製 $n \times n$ 的網格，並處理使用者的滑鼠點擊事件來切換單元格狀態（起點、終點、障礙物）。
2. **後端 (Python / Flask)**：接收前端傳來的網格狀態（維度 n、起點、終點、障礙物位置），生成隨機策略（Random Policy），並計算出每個狀態的價值 $V(s)$。

🟢 HW1-1: 網格地圖開發 (前端與互動)

這部分的重點在於**動態生成網格**與**狀態管理**。

- **表單輸入**：建立一個輸入框限制輸入值為 5 到 9，並有一個「Generate Square」按鈕。
- **動態生成 (JavaScript)**：點擊生成後，利用 JavaScript 在畫面上畫出一個 HTML Table 或 CSS Grid。
- **點擊邏輯 (State Machine)**： 你需要一個陣列或字典來記錄每個格子的狀態（例如 `0: 空白, 1: 起點, 2: 終點, 3: 障礙物`）。 點擊事件的邏輯應該設計為：
  1. 第一次點擊：設為**起點**（CSS 背景改為 `green`）。
  2. 第二次點擊：設為**終點**（CSS 背景改為 `red`）。
  3. 接下來的 $n-2$ 次點擊：設為**障礙物**（CSS 背景改為 `grey`）。

🧠 HW1-2: 策略顯示與價值評估 (後端 Flask 與演算法)

當使用者在前端設定好地圖並按下「計算價值」的按鈕後，利用 AJAX 或 Form Submit 將資料傳給 Flask 後端。

1. 隨機生成行動 (Random Policy)

每個非障礙物、非終點的格子，都需要被賦予一個策略 $\pi(a|s)$。 因為題目要求隨機生成行動，這代表在每個狀態 s 採取上、下、左、右四個方向的機率皆為相等： $$\pi(\text{up}|s) = \pi(\text{down}|s) = \pi(\text{left}|s) = \pi(\text{right}|s) = 0.25$$

2. 策略評估 (Policy Evaluation)

你需要實作 Bellman Expectation Equation 來迭代計算每個狀態的價值 $V(s)$，直到數值收斂。公式如下：

$$V_{k+1}(s) = \sum_{a} \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')]$$

實作演算法的注意事項：

- **初始化**：建立一個 $n \times n$ 的矩陣，將所有狀態的初始值 $V_0(s)$ 設為 0。
- **環境動態 (Transitions)**：
  - 如果朝某個方向移動會撞到「邊界」或「障礙物」，則狀態保持在原地 $s' = s$。
  - 如果到達「終點」，該狀態的價值固定為 0（或依據你課堂的設定給予 Reward），並且不再進行轉移。
- **獎勵 (Reward, r)**：題目沒有明說，但通常 Grid World 每走一步的 Reward 會設為 `-1`，這代表我們希望 Agent 越快走到終點越好。
- **折扣因子 (Discount Factor, γ)**：通常設為 `1.0` 或 `0.9`。

Python 虛擬碼範例：

```
def policy_evaluation(grid_size, start, end, obstacles, theta=0.0001, gamma=1.0):
    # 初始化 V(s) 為 0 的 n x n 矩陣
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

    while True:
        delta = 0
        # 遍歷所有格子
        for i in range(grid_size):
            for j in range(grid_size):
                if (i, j) == end or (i, j) in obstacles:
                    continue # 終點和障礙物不更新 V 值

                v_old = V[i][j]
                v_new = 0

                # 計算四個方向的期望值 (機率皆為 0.25)
                for action in [(0,1), (0,-1), (1,0), (-1,0)]: # 右, 左, 下, 上
                    next_i, next_j = i + action[0], j + action[1]

                    # 邊界或障礙物判定
                    if next_i < 0 or next_i >= grid_size or next_j < 0 or next_j >= grid_size or (next_i, next_j) in obstacles:
                        next_i, next_j = i, j # 留在原地

                    reward = -1 # 假設每走一步 reward 為 -1
                    v_new += 0.25 * (reward + gamma * V[next_i][next_j])

                V[i][j] = v_new
                delta = max(delta, abs(v_old - v_new))

        if delta < theta: # 收斂條件
            break

    return V
```

這個作業的範圍涵蓋了前後端開發以及強化學習演算法的實作。為了能最有效率地幫助你推進進度：

你想先從哪一個部分開始著手？ 我可以先為你撰寫 **HW1-1 的 HTML/JavaScript 網格互動程式碼**，或是提供 **Flask 後端架設與完整 Policy Evaluation 演算法的 Python 程式碼**。

You

先從 index.html 開始生成

Gemini

沒問題！我們先把前端的互動介面與網格地圖（HW1-1）搭建起來。

這裡我為你準備了一個整合了 HTML、CSS 和 JavaScript 的單一 `index.html` 檔案。這支程式會處理表單輸入驗證、動態生成網格，並且實作了「點擊設定起點 $\rightarrow$ 點點設定終點 $\rightarrow$ 點擊設定障礙物」的狀態切換邏輯。

你可以直接建立一個名為 `index.html` 的檔案，然後把以下程式碼貼進去，並在瀏覽器中打開測試：

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Grid World Generator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
            line-height: 1.6;
        }
        .controls {
            margin-bottom: 20px;
        }
        /* 網格容器 */
        #gridContainer {
            display: inline-block;
            border: 2px solid #000;
            margin-top: 20px;
```

```
        }
        .grid-row {
            display: flex;
        }
        .grid-cell {
            width: 50px;
            height: 50px;
            border: 1px solid #333;
            display: flex;
            align-items: center;
            justify-content: center;
            cursor: pointer;
            box-sizing: border-box;
            font-size: 16px;
            user-select: none; /* 防止反白文字 */
        }
        /* 各種單元格狀態的顏色 */
        .cell-start { background-color: green; color: white; font-weight: bold; }
        .cell-end { background-color: red; color: white; font-weight: bold; }
        .cell-obstacle { background-color: grey; color: transparent; } /* 障礙物可以隱藏數字 */

        #instruction {
            font-size: 18px;
            font-weight: bold;
            color: #333;
            margin-top: 10px;
        }
        #calculateBtn {
            margin-top: 20px;
            padding: 10px 20px;
            font-size: 16px;
            background-color: #007BFF;
            color: white;
            border: none;
            cursor: pointer;
            display: none; /* 初始隱藏，設定完才顯示 */
        }
        #calculateBtn:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>

    <h1>Generate n x n Square</h1>

    <div class="controls">
        <label for="gridSize">Enter a number between 5 and 9:</label>
        <input type="number" id="gridSize" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
    </div>

    <h2 id="gridTitle" style="display: none;"></h2>
    <div id="instruction"></div>

    <div id="gridContainer"></div>

    <button id="calculateBtn" onclick="submitToBackend()">計算策略與價值 (Submit to Flask)</button>

    <script>
        // 狀態管理變數
        let n = 0;
        let clickState = 0; // 0: 等待設定起點, 1: 等待設定終點, 2: 等待設定障礙物, 3: 設定完成
        let obstaclesLeft = 0;

        // 儲存地圖資訊，準備傳給後端
        let mapData = {
            size: 0,
            start: null,
            end: null,
            obstacles: []
        };

        function generateGrid() {
            const inputVal = document.getElementById('gridSize').value;
            n = parseInt(inputVal);

            // 驗證輸入範圍
            if (n < 5 || n > 9) {
                alert("Please enter a number between 5 and 9.");
                return;
            }

            // 重置狀態與資料
            clickState = 0;
            obstaclesLeft = n - 2;
            mapData = { size: n, start: null, end: null, obstacles: [] };

            // 更新 UI 標題與提示
            document.getElementById('gridTitle').style.display = 'block';
```

```
        document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
        document.getElementById('calculateBtn').style.display = 'none';
        updateInstruction();

        // 生成網格 DOM
        const container = document.getElementById('gridContainer');
        container.innerHTML = ''; // 清空舊網格

        let cellNumber = 1;
        for (let i = 0; i < n; i++) {
            const rowDiv = document.createElement('div');
            rowDiv.className = 'grid-row';
            for (let j = 0; j < n; j++) {
                const cellDiv = document.createElement('div');
                cellDiv.className = 'grid-cell';
                cellDiv.innerText = cellNumber++;
                // 綁定點擊事件，並傳入座標 (i, j)
                cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
                rowDiv.appendChild(cellDiv);
            }
            container.appendChild(rowDiv);
        }
    }

    function handleCellClick(cellElement, i, j) {
        // 如果這個格子已經被設定過了，就不做任何反應
        if (cellElement.classList.contains('cell-start') ||
            cellElement.classList.contains('cell-end') ||
            cellElement.classList.contains('cell-obstacle')) {
            return;
        }

        if (clickState === 0) {
            // 設定起點 (綠色)
            cellElement.classList.add('cell-start');
            mapData.start = [i, j];
            clickState = 1;
            updateInstruction();
        }
        else if (clickState === 1) {
            // 設定終點 (紅色)
            cellElement.classList.add('cell-end');
            mapData.end = [i, j];
            clickState = 2;
            updateInstruction();
        }
        else if (clickState === 2) {
            // 設定障礙物 (灰色)
            cellElement.classList.add('cell-obstacle');
            mapData.obstacles.push([i, j]);
            obstaclesLeft--;
            updateInstruction();

            // 障礙物設定完畢
            if (obstaclesLeft === 0) {
                clickState = 3;
                document.getElementById('calculateBtn').style.display = 'block'; // 顯示提交按鈕
            }
        }
    }

    function updateInstruction() {
        const inst = document.getElementById('instruction');
        if (clickState === 0) {
            inst.innerText = "1. Click on a cell to set up the start grid as green.";
            inst.style.color = "green";
        } else if (clickState === 1) {
            inst.innerText = "2. Click on a cell to set up the end grid as red.";
            inst.style.color = "red";
        } else if (clickState === 2) {
            inst.innerText = `3. Click on cells to set up obstacles (Grey). ${obstaclesLeft} left.`;
            inst.style.color = "grey";
        } else if (clickState === 3) {
            inst.innerText = "Setup complete! Ready to calculate.";
            inst.style.color = "blue";
        }
    }

    function submitToBackend() {
        // 這個函數是用來將 mapData 透過 AJAX 傳給 Flask 的
        // 目前先用 console.log 讓你看看資料結構
        console.log("準備傳送給後端的資料格式：", mapData);
        alert("請打開 F12 開發者工具的 Console 查看資料格式。下一步我們將把這些資料傳給 Flask！");
    }
</script>

</body>
</html>
```

💡 程式碼亮點說明

1. **狀態機 (State Machine) 管理**：我使用了 `clickState` 變數來控制使用者的點擊流程（起點 $\rightarrow$ 終點 $\rightarrow$ 障礙物）。這可以確保使用者不會在設定起點前就先設定障礙物。
2. **資料結構 `mapData`**：在使用者點擊的過程中，程式會自動將座標 `[i, j]` 記錄在 `mapData` 裡面。這正是我們之後發送 AJAX 請求給 Flask 時，所需的完美 JSON 格式。
3. **動態提示**：隨著步驟的進行，畫面上方的提示文字與顏色會動態改變，完全符合「使用者界面友好性 15%」的評分標準。

打開這個 HTML 檔案玩玩看，確認互動邏輯是不是如你預期。

接下來，你需要我為你撰寫後端的 `app.py` (Flask) 以及接收資料、進行 Policy Evaluation 的演算法部分嗎？

You

灰色格子的數字不見了

Gemini

啊！好眼力。這是我在之前的程式碼裡特意加上去的，不過仔細對照你一開始提供的作業參考圖，灰色的格子確實應該要保留原本的黑色數字才對！

要解決這個問題非常簡單，只需要修改 HTML 檔案中 `<style>` 裡的一行 CSS 即可：

請找到原本的這行程式碼：

```
.cell-obstacle { background-color: grey; color: transparent; } /* 障礙物可以隱藏數字 */
```

將它改成這樣（把 `color` 設為 `black` 或者直接刪掉 `color` 屬性）：

```
.cell-obstacle { background-color: grey; color: black; }
```

存檔並重新整理網頁後，灰色格子裡的數字就會乖乖出現了！

修改完這個小細節後，前端的畫面應該就跟作業的要求一模一樣了。**需要我接著為你產出 Flask 後端 (`app.py`) 的程式碼，把資料傳過去並計算 Policy 和 Value 嗎？**

You

我想要讓他 DEFAULT 設5，一點開就先生成 5x5 grid

Gemini

這個想法很好！這樣使用者一打開網頁就能直接開始點擊，體驗會流暢很多。

要達成這個效果非常簡單，我們只需要在網頁載入完成時，自動呼叫一次 `generateGrid()` 函數就可以了。

請在你的 `index.html` 檔案中，找到 `<script>` 標籤的最底部（也就是 `</script>` 的上一行），加入這段程式碼：

```
        // ... (前面的程式碼保留) ...

        function submitToBackend() {
            console.log("準備傳送給後端的資料格式：", mapData);
            alert("請打開 F12 開發者工具的 Console 查看資料格式。下一步我們將把這些資料傳給 Flask！");
        }

        // 加入這段：當網頁載入完成後，自動觸發生成 5x5 網格
        window.onload = function() {
            generateGrid();
        };
    </script>
```

加上這段之後，只要重新整理網頁，畫面就會預設幫你畫好一個 5x5 的網格，並且直接進入「等待設定起點」的狀態了！

這樣前端的 HW1-1 體驗應該就非常完整了。**準備好進入 Flask 後端與演算法的部分了嗎？**

You

part 1 的前端畫面好了 換 part 2

Gemini

太棒了！前端的基礎已經打好，現在我們進入核心的 **Part 2：Flask 後端與強化學習演算法 (Policy Evaluation)**。

為了完成這部分，我們需要做兩件事：

1. 建立一個 Python 後端 (`app.py`) 來接收前端傳來的網格資料，並執行 **Policy Evaluation (策略評估)** 演算法來算出每個格子的價值 $V(s)$，同時推導出每個格子的策略箭頭。
2. 更新前端的 JavaScript，讓它能把資料傳給後端，並將算出來的結果畫成兩個新的網格（Value Matrix 和 Policy Matrix）。

🟢 步驟 1：建立 Flask 後端 (`app.py`)

請在與 `index.html` 同一個資料夾下，建立一個名為 `app.py` 的檔案。

這個程式會建立一個 `/calculate` 的 API 接口。它會對**隨機策略 (Random Policy, 每個方向機率 25%)** 進行評估，算出價值矩陣後，再找出每個格子的**貪婪策略 (Greedy Policy)** 來決定箭頭方向（這完全符合你上傳的第二張參考圖的效果）。

將以下程式碼貼入 `app.py`：

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # 允許前端跨域請求

def policy_evaluation(grid_size, start, end, obstacles, theta=1e-4, gamma=1.0):
    # 初始化 Value Matrix (V)
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

    # 定義動作：上, 下, 左, 右 (row_change, col_change)
    actions = {
        'up': (-1, 0),
        'down': (1, 0),
        'left': (0, -1),
        'right': (0, 1)
    }

    # 策略評估 (Policy Evaluation) - 評估隨機策略 (每個動作機率 0.25)
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

        for r in range(grid_size):
            for c in range(grid_size):
                state = [r, c]

                # 終點和障礙物的 V 值保持為 0
                if state == end or state in obstacles:
                    continue

                v_expected = 0
                for action_name, (dr, dc) in actions.items():
                    next_r, next_c = r + dr, c + dc

                    # 邊界與障礙物判定 (如果撞牆或撞障礙物，留在原地)
                    if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
                        next_r, next_c = r, c

                    # 獎勵設定：每走一步 reward = -1
                    reward = -1

                    # 隨機策略：每個動作機率 0.25
                    v_expected += 0.25 * (reward + gamma * V[next_r][next_c])

                new_V[r][c] = v_expected
                delta = max(delta, abs(v_expected - V[r][c]))

        V = new_V
        if delta < theta:
            break

    # 推導策略矩陣 (Policy Matrix) - 找出可以得到最大 V 值的方向 (Greedy Policy)
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
            state = [r, c]
            if state == end or state in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            for action_name, (dr, dc) in actions.items():
                next_r, next_c = r + dr, c + dc
                if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
                    next_r, next_c = r, c

                val = -1 + gamma * V[next_r][next_c]

                # 處理浮點數誤差，找出最佳行動
                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [action_name]
                elif abs(val - max_val) <= 1e-6:
                    best_actions.append(action_name)

            policy[r][c] = best_actions

    return V, policy

@app.route('/calculate', methods=['POST'])
def calculate():
```

```
    data = request.json
    grid_size = data['size']
    start = data['start']
    end = data['end']
    obstacles = data['obstacles']

    # 執行演算法
    V, policy = policy_evaluation(grid_size, start, end, obstacles)

    # 將浮點數四捨五入到小數點後兩位，方便前端顯示
    V_rounded = [[round(val, 2) for val in row] for row in V]

    return jsonify({
        'value_matrix': V_rounded,
        'policy_matrix': policy
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

注意： 執行前，請確保你有安裝 Flask 和 Flask-CORS。可以在終端機執行：`pip install flask flask-cors`

🟢 步驟 2：更新前端 `index.html` 的串接邏輯

現在我們需要修改 `index.html` 中的 `submitToBackend()` 函數，讓它能把資料送到 Flask，並將回傳的結果畫成圖表。

請在 `index.html` 的 `<style>` 中**加入**這段 CSS（用來排版結果矩陣）：

```
        .result-container {
            display: flex;
            gap: 40px;
            margin-top: 30px;
        }
        .matrix-wrapper {
            display: flex;
            flex-direction: column;
            align-items: center;
        }
        .matrix-wrapper h3 { margin-bottom: 10px; }
        .arrow { font-size: 20px; font-weight: bold; }
```

接著，在 `<div id="gridContainer"></div>` 和 `<button id="calculateBtn">` 之後，**加入**用來顯示結果的 HTML 區塊：

```
    <div id="results" class="result-container" style="display: none;">
        <div class="matrix-wrapper">
            <h3>Value Matrix</h3>
            <div id="valueGrid"></div>
        </div>
        <div class="matrix-wrapper">
            <h3>Policy Matrix</h3>
            <div id="policyGrid"></div>
        </div>
    </div>
```

最後，**覆蓋**掉原本的 `submitToBackend()` 函數：

```
        async function submitToBackend() {
            try {
                // 發送請求給 Flask
                const response = await fetch('http://127.0.0.1:5000/calculate', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify(mapData)
                });

                const result = await response.json();

                // 顯示結果區塊
                document.getElementById('results').style.display = 'flex';

                // 繪製 Value Matrix 和 Policy Matrix
                renderResultGrid('valueGrid', result.value_matrix, 'value');
                renderResultGrid('policyGrid', result.policy_matrix, 'policy');

            } catch (error) {
                console.error("Error:", error);
                alert("無法連接到後端，請確認 Flask 伺服器 (app.py) 是否正在執行！");
            }
        }

        // 用來繪製結果矩陣的輔助函數
        function renderResultGrid(containerId, matrixData, type) {
            const container = document.getElementById(containerId);
            container.innerHTML = '';

            // 建立箭頭的映射字典
            const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

            for (let i = 0; i < n; i++) {
                const rowDiv = document.createElement('div');
```

```
            rowDiv.className = 'grid-row';
            for (let j = 0; j < n; j++) {
                const cellDiv = document.createElement('div');
                cellDiv.className = 'grid-cell';

                // 判斷是否為特殊格子（終點或障礙物）
                const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
                const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);

                if (isEnd) {
                    cellDiv.classList.add('cell-end');
                    cellDiv.innerText = 'END';
                } else if (isObstacle) {
                    cellDiv.classList.add('cell-obstacle');
                } else {
                    // 填入 Value 或 Policy
                    if (type === 'value') {
                        cellDiv.innerText = matrixData[i][j];
                    } else if (type === 'policy') {
                        // 將 action 陣列轉換成箭頭符號
                        const actionList = matrixData[i][j];
                        cellDiv.innerHTML = `<span class="arrow">${actionList.map(a => arrows[a]).join('')}</span>`;
                    }
                }
                rowDiv.appendChild(cellDiv);
            }
            container.appendChild(rowDiv);
        }
    }
```

🚀 如何執行測試？

1. 打開你的終端機 (Terminal / 命令提示字元)。
2. 進入到這兩個檔案所在的資料夾。
3. 執行 Flask 伺服器：

   ```
   python app.py
   ```

   (你應該會看到 `Running on http://127.0.0.1:5000/` 的提示)
4. 直接用瀏覽器打開你的 `index.html`。
5. 在網頁上依序點擊設定起點、終點、障礙物，然後按下 **"計算策略與價值"**。

這時候，畫面下方就會出現像你作業參考圖一樣的 **Value Matrix (數字)** 與 **Policy Matrix (箭頭)** 了！

你可以先跑跑看，確認運作是否順利。如果需要調整 UI 顏色或是數值計算的細節（例如 Reward 數值的設定），隨時告訴我！

You

幫我改成用 js, 並將 index.html 拆成三個檔案 分別是 html , css, js 檔，並把part 2 寫進 js 中

Gemini

沒問題！完全使用純前端（JavaScript）來執行強化學習演算法也是一個很棒的做法，這樣就不需要架設伺服器，只要打開瀏覽器就能直接算出結果。

我已經幫你把程式碼拆分成三個乾淨的檔案：`index.html`、`style.css` 和 `script.js`，並且將原本 Python 的 Policy Evaluation 演算法完全翻譯成 JavaScript 寫入 `script.js` 中。

請在同一個資料夾下建立這三個檔案，並將以下程式碼分別貼入：

1. `index.html` (網頁結構)

這個檔案現在非常乾淨，只負責 HTML 結構，並引入了 CSS 和 JS 檔案。

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Grid World Generator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>Generate n x n Square</h1>

    <div class="controls">
        <label for="gridSize">Enter a number between 5 and 9:</label>
        <input type="number" id="gridSize" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
    </div>

    <h2 id="gridTitle" style="display: none;"></h2>
    <div id="instruction"></div>

    <div id="gridContainer"></div>
```

```
    <button id="calculateBtn" onclick="calculatePolicy()">計算策略與價值 (Calculate)</button>

    <div id="results" class="result-container" style="display: none;">
        <div class="matrix-wrapper">
            <h3>Value Matrix</h3>
            <div id="valueGrid"></div>
        </div>
        <div class="matrix-wrapper">
            <h3>Policy Matrix</h3>
            <div id="policyGrid"></div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

---

2. `style.css` (外觀樣式)

負責所有的排版與顏色設定。

```
body {
    font-family: Arial, sans-serif;
    margin: 40px;
    line-height: 1.6;
}

.controls {
    margin-bottom: 20px;
}

/* 網格容器 */
#gridContainer, #valueGrid, #policyGrid {
    display: inline-block;
    border: 2px solid #000;
    margin-top: 20px;
    background-color: #fff;
}

.grid-row {
    display: flex;
}

.grid-cell {
    width: 60px;
    height: 60px;
    border: 1px solid #333;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    box-sizing: border-box;
    font-size: 16px;
    user-select: none; /* 防止反白文字 */
    text-align: center;
}

/* 各種單元格狀態的顏色 */
.cell-start { background-color: green; color: white; font-weight: bold; }
.cell-end { background-color: red; color: white; font-weight: bold; }
.cell-obstacle { background-color: grey; color: black; } /* 顯示黑色數字 */

#instruction {
    font-size: 18px;
    font-weight: bold;
    color: #333;
    margin-top: 10px;
}

#calculateBtn {
    margin-top: 20px;
    padding: 10px 20px;
    font-size: 16px;
    background-color: #007BFF;
    color: white;
    border: none;
    cursor: pointer;
    display: none; /* 初始隱藏，設定完才顯示 */
    border-radius: 4px;
}

#calculateBtn:hover {
    background-color: #0056b3;
}

/* 結果矩陣區塊 */
.result-container {
    display: flex;
    gap: 40px;
```

```
    margin-top: 30px;
    flex-wrap: wrap; /* 螢幕不夠寬時自動換行 */
}

.matrix-wrapper {
    display: flex;
    flex-direction: column;
    align-items: center;
}

.matrix-wrapper h3 {
    margin-bottom: 0px;
}

.arrow {
    font-size: 20px;
    font-weight: bold;
    line-height: 1;
}
```

---

3. `script.js` (互動邏輯與演算法)

這裡包含了 UI 的互動邏輯，以及最核心的 Policy Evaluation 演算法（原本在 Flask 裡的部分）。

```
// 狀態管理變數
let n = 0;
let clickState = 0; // 0: 等待設定起點, 1: 等待設定終點, 2: 等待設定障礙物, 3: 設定完成
let obstaclesLeft = 0;

// 儲存地圖資訊
let mapData = {
    size: 0,
    start: null,
    end: null,
    obstacles: []
};

// 網頁載入完成後自動生成 5x5 網格
window.onload = function() {
    generateGrid();
};

function generateGrid() {
    const inputVal = document.getElementById('gridSize').value;
    n = parseInt(inputVal);

    if (n < 5 || n > 9) {
        alert("Please enter a number between 5 and 9.");
        return;
    }

    // 重置狀態與資料
    clickState = 0;
    obstaclesLeft = n - 2;
    mapData = { size: n, start: null, end: null, obstacles: [] };

    // 更新 UI
    document.getElementById('gridTitle').style.display = 'block';
    document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
    document.getElementById('calculateBtn').style.display = 'none';
    document.getElementById('results').style.display = 'none'; // 隱藏之前的結果
    updateInstruction();

    // 生成網格 DOM
    const container = document.getElementById('gridContainer');
    container.innerHTML = '';

    let cellNumber = 1;
    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';
            cellDiv.innerText = cellNumber++;
            cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}

function handleCellClick(cellElement, i, j) {
    if (cellElement.classList.contains('cell-start') ||
        cellElement.classList.contains('cell-end') ||
        cellElement.classList.contains('cell-obstacle')) {
        return;
    }
```

```
    if (clickState === 0) {
        cellElement.classList.add('cell-start');
        mapData.start = [i, j];
        clickState = 1;
        updateInstruction();
    }
    else if (clickState === 1) {
        cellElement.classList.add('cell-end');
        mapData.end = [i, j];
        clickState = 2;
        updateInstruction();
    }
    else if (clickState === 2) {
        cellElement.classList.add('cell-obstacle');
        mapData.obstacles.push([i, j]);
        obstaclesLeft--;
        updateInstruction();

        if (obstaclesLeft === 0) {
            clickState = 3;
            document.getElementById('calculateBtn').style.display = 'block';
        }
    }
}

function updateInstruction() {
    const inst = document.getElementById('instruction');
    if (clickState === 0) {
        inst.innerText = "1. Click on a cell to set up the start grid as green.";
        inst.style.color = "green";
    } else if (clickState === 1) {
        inst.innerText = "2. Click on a cell to set up the end grid as red.";
        inst.style.color = "red";
    } else if (clickState === 2) {
        inst.innerText = `3. Click on cells to set up obstacles (Grey). ${obstaclesLeft} left.`;
        inst.style.color = "grey";
    } else if (clickState === 3) {
        inst.innerText = "Setup complete! Ready to calculate.";
        inst.style.color = "blue";
    }
}

// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() {
    const gridSize = mapData.size;
    const end = mapData.end;
    const obstacles = mapData.obstacles;

    const theta = 1e-4;
    const gamma = 1.0;

    // 初始化 V 為 0
    let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));
    const actions = {
        'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1]
    };

    // 輔助函數：判斷是否為障礙物或終點
    const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c);
    const isEnd = (r, c) => end[0] === r && end[1] === c;

    // 1. 執行 Policy Evaluation (隨機策略 0.25)
    while (true) {
        let delta = 0;
        let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));

        for (let r = 0; r < gridSize; r++) {
            for (let c = 0; c < gridSize; c++) {
                if (isEnd(r, c) || isObstacle(r, c)) continue;

                let vExpected = 0;
                for (let action in actions) {
                    let nextR = r + actions[action][0];
                    let nextC = c + actions[action][1];

                    // 撞牆或障礙物則留在原地
                    if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {
                        nextR = r;
                        nextC = c;
                    }

                    let reward = -1; // 每走一步的 reward
                    vExpected += 0.25 * (reward + gamma * V[nextR][nextC]);
                }
                newV[r][c] = vExpected;
                delta = Math.max(delta, Math.abs(vExpected - V[r][c]));
            }
        }
        V = newV;
        if (delta < theta) break; // 收斂
```

```
    }

    // 2. 找出 Greedy Policy (推導箭頭方向)
    let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([]));

    for (let r = 0; r < gridSize; r++) {
        for (let c = 0; c < gridSize; c++) {
            if (isEnd(r, c) || isObstacle(r, c)) continue;

            let maxVal = -Infinity;
            let bestActions = [];

            for (let action in actions) {
                let nextR = r + actions[action][0];
                let nextC = c + actions[action][1];

                if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {
                    nextR = r;
                    nextC = c;
                }

                let val = -1 + gamma * V[nextR][nextC];

                // 處理浮點數誤差
                if (val > maxVal + 1e-6) {
                    maxVal = val;
                    bestActions = [action];
                } else if (Math.abs(val - maxVal) <= 1e-6) {
                    bestActions.push(action); // 多個方向同等好
                }
            }
            policy[r][c] = bestActions;
        }
    }

    // 將 V 的數值四捨五入到小數點後兩位
    let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

    // 顯示結果
    document.getElementById('results').style.display = 'flex';
    renderResultGrid('valueGrid', roundedV, 'value');
    renderResultGrid('policyGrid', policy, 'policy');
}

// 繪製結果圖表的函數
function renderResultGrid(containerId, matrixData, type) {
    const container = document.getElementById(containerId);
    container.innerHTML = '';

    const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';

            const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);

            if (isEnd) {
                cellDiv.classList.add('cell-end');
                // 可自行決定終點要顯示 END 還是空白
            } else if (isObstacle) {
                cellDiv.classList.add('cell-obstacle');
            } else {
                if (type === 'value') {
                    cellDiv.innerText = matrixData[i][j];
                } else if (type === 'policy') {
                    // 將 action 陣列轉換成多個箭頭
                    const actionList = matrixData[i][j];
                    const arrowStr = actionList.map(a => arrows[a]).join('');
                    cellDiv.innerHTML = `<span class="arrow">${arrowStr}</span>`;
                }
            }
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}
```

這樣一來，你的作業就完全變成了輕量級的網頁應用程式，不依賴任何後端伺服器！直接用瀏覽器打開 `index.html` 就可以測試完整的設定、點擊到計算出矩陣結果的流程。

需要我幫忙解釋演算法裡 `V = newV` 收斂邏輯，或者調整箭頭顯示的大小和樣式嗎？

You

// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() { const gridSize = mapData.size; const end = mapData.end; const obstacles = mapData.obstacles; const theta = 1e-4; const gamma = 1.0; // 初始化 V 為 0 let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); const actions = { 'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1] };

// 輔助函數：判斷是否為障礙物或終點 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25) while (true) { let delta = 0; let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let vExpected = 0; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; // 撞牆或障礙物則留在原地 if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let reward = -1; // 每走一步的 reward vExpected += 0.25 * (reward + gamma * V[nextR][nextC]); } newV[r][c] = vExpected; delta = Math.max(delta, Math.abs(vExpected - V[r][c])); } } V = newV; if (delta < theta) break; // 收斂 }

// 2. 找出 Greedy Policy (推導箭頭方向) let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([])); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let maxVal = -Infinity; let bestActions = []; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let val = -1 + gamma * V[nextR][nextC]; // 處理浮點數誤差 if (val > maxVal + 1e-6) { maxVal = val; bestActions = [action]; } else if (Math.abs(val - maxVal) <= 1e-6) { bestActions.push(action); // 多個方向同等好 } } policy[r][c] = bestActions; } }

// 將 V 的數值四捨五入到小數點後兩位 let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果 document.getElementById('results').style.display = 'flex'; renderResultGrid('valueGrid', roundedV, 'value'); renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) { const container = document.getElementById(containerId); container.innerHTML = ''; const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) { const rowDiv = document.createElement('div'); rowDiv.className = 'grid-row'; for (let j = 0; j < n; j++) { const cellDiv = document.createElement('div'); cellDiv.className = 'grid-cell'; const isEnd = (mapData.end[0] === i && mapData.end[1] === j); const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j); if (isEnd) { cellDiv.classList.add('cell-end'); // 可自行決定終點要顯示 END 還是空白 } else if (isObstacle) { cellDiv.classList.add('cell-obstacle'); } else { if (type === 'value') { cellDiv.innerText = matrixData[i][j]; } else if (type === 'policy') { // 將 action 陣列轉換成多個箭頭 const actionList = matrixData[i][j]; const arrowStr = actionList.map(a => arrows[a]).join(''); cellDiv.innerHTML = ${arrowStr}; } } rowDiv.appendChild(cellDiv); } container.appendChild(rowDiv); } }

解釋這段的目的與程式碼內容// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() { const gridSize = mapData.size; const end = mapData.end; const obstacles = mapData.obstacles; const theta = 1e-4; const gamma = 1.0; // 初始化 V 為 0 let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); const actions = { 'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1] };

// 輔助函數：判斷是否為障礙物或終點 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25) while (true) { let delta = 0; let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let vExpected = 0; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; // 撞牆或障礙物則留在原地 if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let reward = -1; // 每走一步的 reward vExpected += 0.25 * (reward + gamma * V[nextR][nextC]); } newV[r][c] = vExpected; delta = Math.max(delta, Math.abs(vExpected - V[r][c])); } } V = newV; if (delta < theta) break; // 收斂 }

// 2. 找出 Greedy Policy (推導箭頭方向) let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([])); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let maxVal = -Infinity; let bestActions = []; for (let action in actions) { let nextR = r + actions[action][0]; let nextC = c + actions[action][1]; if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) { nextR = r; nextC = c; } let val = -1 + gamma * V[nextR][nextC]; // 處理浮點數誤差 if (val > maxVal + 1e-6) { maxVal = val; bestActions = [action]; } else if (Math.abs(val - maxVal) <= 1e-6) { bestActions.push(action); // 多個方向同等好 } } policy[r][c] = bestActions; } }

// 將 V 的數值四捨五入到小數點後兩位 let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果 document.getElementById('results').style.display = 'flex'; renderResultGrid('valueGrid', roundedV, 'value'); renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) { const container = document.getElementById(containerId); container.innerHTML = ''; const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) { const rowDiv = document.createElement('div'); rowDiv.className = 'grid-row'; for (let j = 0; j < n; j++) { const cellDiv = document.createElement('div'); cellDiv.className = 'grid-cell'; const isEnd = (mapData.end[0] === i && mapData.end[1] === j); const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j); if (isEnd) { cellDiv.classList.add('cell-end'); // 可自行決定終點要顯示 END 還是空白 } else if (isObstacle) { cellDiv.classList.add('cell-obstacle'); } else { if (type === 'value') { cellDiv.innerText = matrixData[i][j]; } else if (type === 'policy') { // 將 action 陣列轉換成多個箭頭 const actionList = matrixData[i][j]; const arrowStr = actionList.map(a => arrows[a]).join(''); cellDiv.innerHTML = ${arrowStr}; } } rowDiv.appendChild(cellDiv); } container.appendChild(rowDiv); } }

解釋這段的目的與程式碼內容 // === HW1-2: 策略評估演算法 (JavaScript 實作) ===

function calculatePolicy() { const gridSize = mapData.size; const end = mapData.end; const obstacles = mapData.obstacles; const theta = 1e-4; const gamma = 1.0; // 初始化 V 為 0 let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); const actions = { 'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1] };

// 輔助函數：判斷是否為障礙物或終點 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25) while (true) { let delta = 0; let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0)); for (let r = 0; r < gridSize; r++) { for (let c = 0; c < gridSize; c++) { if (isEnd(r, c) || isObstacle(r, c)) continue; let vExpected = 0;

```
            for (let action in actions) {                    let nextR = r + actions[action][0];                    let nextC = c + actions[action][1];                                    // 撞牆或障礙物則留在原地                    if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                        nextR = r;
nextC = c;                    }                                    let reward = -1; // 每走一步的 reward                    vExpected += 0.25 * (reward + gamma * V[nextR][nextC]);
    }                newV[r][c] = vExpected;                    delta = Math.max(delta, Math.abs(vExpected - V[r][c]));                }        }        V = newV;        if (delta < theta) break; // 收斂    }

// 2. 找出 Greedy Policy (推導箭頭方向)    let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([]));        for (let r = 0; r < gridSize; r++) {        for (let c = 0; c < gridSize; c++) {            if (isEnd(r, c) || isObstacle(r, c)) continue;                    let maxVal = -Infinity;            let bestActions = [];                    for (let action in actions) {                let nextR = r + actions[action][0];                let nextC = c + actions[action][1];                                if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                    nextR = r;                    nextC = c;                }                                let val = -1 + gamma * V[nextR][nextC];                                // 處理浮點數誤差                if (val > maxVal + 1e-6) {                    maxVal = val;                    bestActions = [action];                } else if (Math.abs(val - maxVal) <= 1e-6) {                    bestActions.push(action); // 多個方向同等好                }            }            policy[r][c] = bestActions;        }    }

// 將 V 的數值四捨五入到小數點後兩位    let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果    document.getElementById('results').style.display = 'flex';    renderResultGrid('valueGrid', roundedV, 'value');    renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) {    const container = document.getElementById(containerId);    container.innerHTML = '';        const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) {        const rowDiv = document.createElement('div');        rowDiv.className = 'grid-row';        for (let j = 0; j < n; j++) {            const cellDiv = document.createElement('div');            cellDiv.className = 'grid-cell';                const isEnd = (mapData.end[0] === i && mapData.end[1] === j);            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);                if (isEnd) {                cellDiv.classList.add('cell-end');                // 可自行決定終點要顯示 END 還是空白            } else if (isObstacle) {                cellDiv.classList.add('cell-obstacle');            } else {                if (type === 'value') {
  cellDiv.innerText = matrixData[i][j];                } else if (type === 'policy') {                    // 將 action 陣列轉換成多個箭頭                    const actionList = matrixData[i][j];                    const arrowStr = actionList.map(a => arrows[a]).join('');                    cellDiv.innerHTML = <span class="arrow">${arrowStr}</span>;                }
    }            rowDiv.appendChild(cellDiv);        }        container.appendChild(rowDiv);    } }
```

解釋這段的目的與程式碼內容// === HW1-2: 策略評估演算法 (JavaScript 實作) ===

```
function calculatePolicy() {    const gridSize = mapData.size;    const end = mapData.end;    const obstacles = mapData.obstacles;        const theta = 1e-4;    const gamma = 1.0;        // 初始化 V 為 0    let V = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));    const actions = {        'up': [-1, 0], 'down': [1, 0], 'left': [0, -1], 'right': [0, 1]    };

// 輔助函數：判斷是否為障礙物或終點    const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c);    const isEnd = (r, c) => end[0] === r && end[1] === c;

// 1. 執行 Policy Evaluation (隨機策略 0.25)    while (true) {        let delta = 0;        let newV = Array.from({ length: gridSize }, () => Array(gridSize).fill(0.0));
for (let r = 0; r < gridSize; r++) {            for (let c = 0; c < gridSize; c++) {                if (isEnd(r, c) || isObstacle(r, c)) continue;                    let vExpected = 0;
            for (let action in actions) {                    let nextR = r + actions[action][0];                    let nextC = c + actions[action][1];                                    // 撞牆或障礙物則留在原地                    if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                        nextR = r;
nextC = c;                    }                                    let reward = -1; // 每走一步的 reward                    vExpected += 0.25 * (reward + gamma * V[nextR][nextC]);
    }                newV[r][c] = vExpected;                    delta = Math.max(delta, Math.abs(vExpected - V[r][c]));                }        }        V = newV;        if (delta < theta) break; // 收斂    }

// 2. 找出 Greedy Policy (推導箭頭方向)    let policy = Array.from({ length: gridSize }, () => Array(gridSize).fill([]));        for (let r = 0; r < gridSize; r++) {        for (let c = 0; c < gridSize; c++) {            if (isEnd(r, c) || isObstacle(r, c)) continue;                    let maxVal = -Infinity;            let bestActions = [];                    for (let action in actions) {                let nextR = r + actions[action][0];                let nextC = c + actions[action][1];                                if (nextR < 0 || nextR >= gridSize || nextC < 0 || nextC >= gridSize || isObstacle(nextR, nextC)) {                    nextR = r;                    nextC = c;                }                                let val = -1 + gamma * V[nextR][nextC];                                // 處理浮點數誤差                if (val > maxVal + 1e-6) {                    maxVal = val;                    bestActions = [action];                } else if (Math.abs(val - maxVal) <= 1e-6) {                    bestActions.push(action); // 多個方向同等好                }            }            policy[r][c] = bestActions;        }    }

// 將 V 的數值四捨五入到小數點後兩位    let roundedV = V.map(row => row.map(val => Number(val.toFixed(2))));

// 顯示結果    document.getElementById('results').style.display = 'flex';    renderResultGrid('valueGrid', roundedV, 'value');    renderResultGrid('policyGrid', policy, 'policy'); }

// 繪製結果圖表的函數 function renderResultGrid(containerId, matrixData, type) {    const container = document.getElementById(containerId);    container.innerHTML = '';        const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) {        const rowDiv = document.createElement('div');        rowDiv.className = 'grid-row';        for (let j = 0; j < n; j++) {            const cellDiv = document.createElement('div');            cellDiv.className = 'grid-cell';                const isEnd = (mapData.end[0] === i && mapData.end[1] === j);            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);                if (isEnd) {                cellDiv.classList.add('cell-end');                // 可自行決定終點要顯示 END 還是空白            } else if (isObstacle) {                cellDiv.classList.add('cell-obstacle');            } else {                if (type === 'value') {
  cellDiv.innerText = matrixData[i][j];                } else if (type === 'policy') {                    // 將 action 陣列轉換成多個箭頭                    const actionList = matrixData[i][j];                    const arrowStr = actionList.map(a => arrows[a]).join('');                    cellDiv.innerHTML = <span class="arrow">${arrowStr}</span>;                }
    }            rowDiv.appendChild(cellDiv);        }        container.appendChild(rowDiv);    } }
```

解釋這段的目的與程式碼內容

Gemini

這段程式碼是整個作業最核心的「大腦」與「畫筆」。它的主要目的是**利用強化學習（Reinforcement Learning）的演算法來計算出每個格子的價值，並把結果畫在網頁上**。

我可以將這段程式碼拆分成兩個主要函數來為你詳細解釋：

🧠 1. `calculatePolicy()`：計算策略與價值（核心演算法）

這個函數負責執行強化學習中的 **Policy Evaluation（策略評估）**，用來推導出數字矩陣（Value Matrix）與箭頭矩陣（Policy Matrix）。它包含了以下幾個重要階段：

- **初始化設定**：
  - `theta = 1e-4`：這是「收斂閾值」。當數值更新的幅度小於這個值時，就代表計算完成了。
  - `gamma = 1.0`：這是「折扣因子（Discount Factor）」，決定未來獎勵的重要性（1.0 代表未來與現在的獎勵一樣重要）。
  - `V`：建立一個 $n \times n$ 的陣列，將所有格子的初始價值設為 0。
- **第一階段：執行 Policy Evaluation（評估隨機策略）**：

    - 這裡使用了一個 `while (true)` 迴圈，不斷迭代更新每個格子的價值 $V(s)$，直到數值不再產生明顯變化（`delta < theta`）為止。
    - 它假設在每個格子裡，往上、下、左、右走的機率都是 **0.25**（隨機策略）。
    - **撞牆判定**：如果移動後會超出邊界或撞到障礙物，程式會讓它「留在原地」（`nextR = r; nextC = c;`）。
    - **獎勵計算**：每走一步的代價（Reward）固定為 `-1`。它利用貝爾曼期望方程式（Bellman Expectation Equation）不斷累加期望值：`vExpected += 0.25 * (reward + gamma * V[nextR][nextC])`。
- **第二階段：找出 Greedy Policy（推導最佳箭頭方向）**：
    - 當所有格子的價值 V 都算好並收斂後，程式會再掃描一次地圖。
    - 這次的目的是「看哪個方向的未來價值最高」。它會測試四個方向，找出能得到最大 `val` 的行動（Action）。
    - 為了避免 JavaScript 浮點數計算的微小誤差，這裡加上了 `1e-6` 的容錯範圍。如果有多個方向的數值一樣好，就會把這些方向都存進 `bestActions` 裡（這就是為什麼有些格子會有雙向或三向箭頭）。
- **輸出準備**：最後，它會將計算出來的 V 值四捨五入到小數點後兩位，並呼叫第二個函數來畫圖。

🎨 2. `renderResultGrid()`：繪製結果圖表（畫面渲染）

這個函數是一個「共用畫筆」，它的目的是**接收剛剛算出來的數學矩陣，並把它們變成網頁上肉眼可見的方格與文字**。

- **動態建立 DOM 元素**：它會根據網格大小 n，利用兩個 `for` 迴圈動態生成 HTML 的 `<div>` 標籤來組成網格。
- **狀態著色**：
    - 如果是終點，它會套用 `cell-end` 的 CSS 樣式（紅色背景）。
    - 如果是障礙物，套用 `cell-obstacle`（灰色背景）。
- **填寫內容（根據 `type` 決定）**：
    - 如果傳入的 `type` 是 `'value'`：它會把陣列裡的小數數字直接填入格子裡（對應 Value Matrix）。
    - 如果傳入的 `type` 是 `'policy'`：它會拿出剛剛算出的 `bestActions` 陣列（例如 `['up', 'right']`），透過 `arrows` 字典把它們轉換成真正的箭頭符號（如 ↑→），並顯示在格子裡。

總結來說：這段程式碼完美取代了原本需要由 Python 後端來做的事情。它用 JavaScript 在使用者的瀏覽器裡直接把「走到終點的最短預期步數（Value）」算出來，並指出了「最聰明的走法方向（Policy）」。

這部分的邏輯比較吃重數學基礎。**你對程式碼裡的「貝爾曼方程式計算（`vExpected`）」或是「邊界撞擊處理」有哪裡需要我進一步舉例說明的嗎？**

You

寫一個 readme.md 內容包含 demo網址 https://bibaisafish.github.io/DRL_HW1/

Gemini

這是一份為你的專案量身打造的 `README.md` 檔案。內容涵蓋了作業要求、功能介紹、操作步驟，並且把你的 Demo 網址放在了最顯眼的位置，讓教授或助教點進去就能馬上看到成品！

你可以直接複製以下內容，貼到你的 `README.md` 檔案中：

```
# Deep Reinforcement Learning - HW1: Grid World Policy Evaluation

這是一個基於純前端技術 (HTML, CSS, JavaScript) 開發的「網格世界 (Grid World)」強化學習互動式網頁應用程式。本專案實作了隨機策略下的**策略評估 (Poli

👉 **[點此觀看 Live Demo](https://bibaisafish.github.io/DRL_HW1/)**

---

## 🌟 專案功能 (Features)

本專案分為兩個主要部分，完全符合 HW1 的作業要求：

### HW1-1: 網格地圖開發 (前端互動)
* **動態網格生成**：允許用戶自由輸入維度 `n` (範圍從 5 到 9)，動態生成 `n x n` 的網格地圖。
* **狀態機互動設計**：透過流暢的滑鼠點擊流程設定環境：
  1. **起點 (Start)**：點擊設定為綠色單元格。
  2. **終點 (End)**：點擊設定為紅色單元格。
  3. **障礙物 (Obstacles)**：設定 `n-2` 個障礙物，點擊變為灰色單元格（保留原始數字顯示）。

### HW1-2: 策略顯示與價值評估 (核心演算法)
* **策略評估 (Policy Evaluation)**：基於隨機策略（上、下、左、右機率各 25%），透過貝爾曼期望方程式 (Bellman Expectation Equation) 迭代計算出每個此
* **策略矩陣 (Policy Matrix)**：根據算出的 `V(s)` 推導出 Greedy Policy，在畫面上顯示每個單元格的最佳行動方向（↑, ↓, ←, →），若有多個最佳方向亦會同
* **價值矩陣 (Value Matrix)**：清楚呈現每個單元格最終收斂的預期價值（四捨五入至小數點後兩位）。

---

## 🛠 技術棧 (Tech Stack)

* **HTML5**：架構網頁主體與矩陣容器。
* **CSS3**：使用 Flexbox 排版，設計直觀的 UI 介面與狀態顏色。
* **Vanilla JavaScript (ES6)**：
  * 處理 DOM 操作與滑鼠點擊狀態機。
  * 實作 Reinforcement Learning 的 Policy Evaluation 演算法（無須後端伺服器即可在瀏覽器端完成矩陣運算）。

---

## 🚀 如何使用 (How to Use)
```

```
1. 開啟 [Demo 網頁](https://bibaisafish.github.io/DRL_HW1/)。
2. 在左上角的輸入框輸入 5 到 9 之間的數字，並點擊 **"Generate Square"** 生成網格。
3. 根據畫面上方的提示，依序點擊網格來設定：
   * 第 1 下：設定**起點** (綠色)。
   * 第 2 下：設定**終點** (紅色)。
   * 接下來的 `n-2` 下：設定**障礙物** (灰色)。
4. 設定完成後，點擊下方出現的 **"計算策略與價值 (Calculate)"** 按鈕。
5. 頁面下方會立刻生成並顯示計算完成的 **Value Matrix** (數值) 與 **Policy Matrix** (方向箭頭)。

---

## 📌 演算法參數設定 (Hyperparameters)

* **Reward (獎勵)**：每移動一步的 reward 為 `-1` (目標為尋找最短路徑)。
* **Discount Factor ($\gamma$)**：`1.0`
* **Convergence Threshold ($\theta$)**：`1e-4`
* **Policy**：隨機策略 (Random Policy)，每個方向的轉移機率為 `0.25`。撞牆或撞障礙物則狀態留在原地。
```

這份 README 結構清晰，而且將技術亮點（無後端、純 JS 運算）和演算法參數都標示了出來，這在交作業時通常能讓評分者一目了然。

你覺得這個 README 的排版和內容可以嗎？需要我教你怎麼用 Git 把這些檔案推送到 GitHub Pages 上嗎？

You

如果我現在拆成 .html, .css, .js, .py 這四個檔案並丟到同一個 repository 下，stream lit web 可以成功運行嗎 .html 會用到 .py

---

Gemini

沒辦法直接這樣運作喔！ 這是一個很常見的架構迷思，我來為你釐清一下 Streamlit 的運作邏輯。

如果你把 `.html`, `.css`, `.js`, `.py` 丟到同一個 Repository，然後交給 Streamlit Cloud 部署，**它不會照你預期的方式運作，你的 `.html` 無法直接「呼叫」或「使用」那個 `.py` 檔。**

🔍 為什麼不行？(Streamlit 的運作機制)

傳統的網頁開發（像是我們之前提到的 Flask）和 Streamlit 有著根本上的架構差異：

1. **傳統架構 (如 Flask / FastAPI)：** 前後端是分離的。你的 `.html` (配合 `.js`) 是主角，當使用者點擊按鈕時，`.js` 會透過 `fetch()` 或 AJAX 發送網路請求 (API) 給後端的 `.py`，後端算完再把資料傳回前端。
2. **Streamlit 架構： `.py` 才是唯一的主角**。Streamlit 的設計初衷是讓資料科學家「完全不需要寫 HTML/JS/CSS」，只要寫 Python 程式碼，Streamlit 就會在底層自動幫你生成網頁。
   - Streamlit **不能**當作 API 伺服器來接收前端的 `fetch()` 請求。
   - 如果你在資料夾裡放了一個 `index.html`，Streamlit Cloud 基本上會忽視它，它只會去執行你的 Python 檔案（例如 `app.py`）。

雖然 Streamlit 有一個進階功能叫 "Streamlit Components"（可以嵌入自訂的 HTML/JS），但要讓 JS 的點擊狀態與 Python 的運算結果互相溝通，寫起來非常複雜，完全失去了 Streamlit 「簡單快速」的優勢，對這個作業來說算是殺雞用牛刀了。

💡 你現在最好的做法是什麼？

既然你已經有了完美的 `.html`, `.css`, `.js` (純前端版本)，而且你連 GitHub Pages 的網址 (`https://bibaisafish.github.io/DRL_HW1/`) 都設好了，我強烈建議你：

👉 **方案 A：維持現狀，使用「純前端 (純 JS) 版本」 + GitHub Pages** 這是最完美的解法！你**完全不需要 `.py` 檔，也不需要 Streamlit**。我們剛剛寫好的 `script.js` 已經包含了完整的強化學習演算法。你只要把 `.html`, `.css`, `.js` 這三個檔案推送到 GitHub，然後開啟 GitHub Pages 功能，你的作業就大功告成了！教授點開網址就能直接玩，完全不用等待伺服器喚醒。

👉 **方案 B：如果你「規定」一定要用 Python 寫演算法** 如果你教授規定後端演算法一定要用 Python 實作，那你不能用 Streamlit。你必須回到我們一開始討論的 **Flask** 架構。

- 你需要把 `.html`, `.css`, `.js` 和寫著 Flask 路由的 `.py` 包在一起。
- 不能部署在 Streamlit Cloud 或 GitHub Pages 上。你需要將它部署到支援 Web API 的平台，例如 **Render**、**Zeabur** 或 **Heroku**。

所以，你目前的作業要求中，有強制規定演算法必須用 Python 寫嗎？還是只要網頁能正確跑出結果就好呢？

You

重新寫一個 app.py 會更好嗎

Gemini

這完全取決於**「教授或助教的評分標準」**！

要判斷重新寫一個 `app.py` 會不會更好，我們需要先釐清你這堂課（強化學習）的作業要求。以下我幫你分析兩種情況，你可以評估看看哪一種最適合你：

情況一：教授規定「核心演算法必須用 Python 實作」🐍

如果這堂課非常注重 Python 程式碼，助教甚至會打開你的程式碼來檢查你是怎麼寫 Policy Evaluation 的，那你**確實需要一個 `app.py`**。

但在這種情況下，我不建議你用 Streamlit，我強烈建議你用 **Flask**：

- **為什麼不用 Streamlit？** Streamlit 雖然架設簡單，但它的網頁是「資料驅動」的。你要實作「點第一下變綠色、點第二下變紅色、接著點變灰色」這種**精細的滑鼠互動狀態機，在 Streamlit 裡會是一場災難**（因為每次點擊都會導致整個畫面重新載入）。
- **更好的做法 (Flask)**：也就是我們一開始討論的架構。前端維持你漂亮的 HTML/JS/CSS，後端寫一個 Flask 的 `app.py`。
  - **代價**：你不能用 GitHub Pages 或 Streamlit Cloud 部署了。你需要把這四個檔案一起丟到 Render 或 Zeabur 這類支援 Python Web Server 的免費平台上。

情況二：教授只看「網頁操作流暢度與演算法結果是否正確」✨

如果作業說明（如你之前貼的評分標準）只寫了「網頁操作流暢度 5%」、「策略評估的正確性 15%」，而沒有強制規定語言，那麼**維持現在的「純 JS 前端版本」絕對是最好的選擇！**

- **為什麼更好？**
  1. **零延遲**：所有的矩陣運算都在使用者的瀏覽器裡瞬間完成，體驗極佳。
  2. **部署超簡單**：你已經有了 `https://bibaisafish.github.io/DRL_HW1/` 這個 GitHub Pages 網址，這對交作業來說是最穩定、最不會出包的展示方式（不需要擔心 Demo 時後端伺服器睡著或斷線）。
  3. **架構乾淨**：不需要維護前後端的 API 溝通。

💡 我的建議與下一步

如果你確定**不需要**交 Python 檔，那我建議你就用現在的純 HTML/JS/CSS 版本直接交差，這個版本的完成度已經非常高了！

但如果你心裡不踏實，或者教授有明文規定「請上傳 .py 檔」： 請告訴我，我會立刻幫你把架構切換回 **Flask 的 `app.py`**，並教你怎麼把它免費部署到 Render 上，這樣你就能同時保有「順暢的網頁互動」與「Python 演算法的後端」了！

你覺得哪一種情況比較符合你們的作業規定呢？

You

我會想改 app.py 是因為現在有新的 hw1-3 了

HW1-3: 使用價值迭代算法推導最佳政策 價值迭代算法： 實現價值迭代算法來計算最佳政策。 使用該算法推導每個格子的最佳行動。 最佳政策顯示： 顯示推導出的最佳政策，通過顯示每個狀態（格子）對應的行動來展示最佳政策。這些行動應該取代之前顯示的隨機行動。 顯示價值函數： 在執行價值迭代後，更新格子以顯示每個狀態的價值函數 V(s)。 每個格子應顯示對應的值，表示在最佳政策下該狀態的期望回報。 其他注意事項： 使用 Flask 作為後端，HTML/CSS 作為前端來顯示格子並處理用戶交互。 確保 Flask 應用程序是交互式的，用戶可以點擊格子來設置起始點、終點、障礙物，並查看生成的政策。 提供可視化的功能，讓用戶能夠清楚地看到隨機政策、價值函數和最佳政策的變化。

我會想改 app.py 是因為現在有新的 hw1-3 了

HW1-3: 使用價值迭代算法推導最佳政策 價值迭代算法： 實現價值迭代算法來計算最佳政策。 使用該算法推導每個格子的最佳行動。 最佳政策顯示： 顯示推導出的最佳政策，通過顯示每個狀態（格子）對應的行動來展示最佳政策。這些行動應該取代之前顯示的隨機行動。 顯示價值函數： 在執行價值迭代後，更新格子以顯示每個狀態的價值函數 V(s)。 每個格子應顯示對應的值，表示在最佳政策下該狀態的期望回報。 其他注意事項： 使用 Flask 作為後端，HTML/CSS 作為前端來顯示格子並處理用戶交互。 確保 Flask 應用程序是交互式的，用戶可以點擊格子來設置起始點、終點、障礙物，並查看生成的政策。 提供可視化的功能，讓用戶能夠清楚地看到隨機政策、價值函數和最佳政策的變化。

Gemini

原來如此！既然作業 **HW1-3 已經明確規定「使用 Flask 作為後端」**，而且要求你實作 **Value Iteration (價值迭代算法)** 來和之前的隨機策略做對比，那我們就**必須把架構改回 Python Flask + HTML/JS/CSS 的模式**了。

這個作業的設計非常經典，它的目的是讓你比較兩種演算法：

1. **HW1-2 策略評估 (Policy Evaluation)**：評估「隨機亂走」的表現有多差，並推導出基於這個爛表現的策略。
2. **HW1-3 價值迭代 (Value Iteration)**：直接用 `max` 找出「最完美」的走法 (最佳政策) 與預期價值。

為了讓你達成「可視化比較」的要求，我幫你設計了完整的四個檔案，讓畫面上同時顯示 **HW1-2 的結果**與 **HW1-3 的結果**，教授一看就能給高分！

以下是你需要的 4 個檔案完整程式碼：

1. `app.py` (Flask 後端與兩種演算法)

這個檔案包含了 HW1-2 和 HW1-3 的兩種演算法。請先確保有安裝套件：`pip install flask flask-cors`

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # 允許跨域請求

# 定義行動方向
ACTIONS = {'up': (-1, 0), 'down': (1, 0), 'left': (0, -1), 'right': (0, 1)}

def get_next_state(r, c, action_name, grid_size, obstacles):
    dr, dc = ACTIONS[action_name]
    next_r, next_c = r + dr, c + dc
    # 如果撞牆或撞障礙物，留在原地
    if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
        return r, c
    return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0):
```

```
    """根據 V 值推導出最佳政策 (Greedy Policy)"""
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
            if [r, c] == end or [r, c] in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            for a in ACTIONS:
                nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                val = -1 + gamma * V[nr][nc]

                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [a]
                elif abs(val - max_val) <= 1e-6:
                    best_actions.append(a)

            policy[r][c] = best_actions
    return policy

# === HW1-2: 策略評估 (Random Policy) ===
def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue
                v_expected = 0
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    v_expected += 0.25 * (-1 + gamma * V[nr][nc])

                new_V[r][c] = v_expected
                delta = max(delta, abs(v_expected - V[r][c]))
        V = new_V
        if delta < theta: break

    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

# === HW1-3: 價值迭代 (Value Iteration) ===
def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue

                max_v = -float('inf')
                # Value Iteration: 取四個方向中能帶來"最大價值"的動作 (取代原本的 *0.25)
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    v = -1 + gamma * V[nr][nc]
                    if v > max_v:
                        max_v = v

                new_V[r][c] = max_v
                delta = max(delta, abs(max_v - V[r][c]))
        V = new_V
        if delta < theta: break

    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    size = data['size']
    end = data['end']
    obstacles = data['obstacles']

    # 計算兩種演算法的結果
    v_hw2, p_hw2 = policy_evaluation(size, end, obstacles)
    v_hw3, p_hw3 = value_iteration(size, end, obstacles)

    # 數值四捨五入
    v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2]
    v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3]

    return jsonify({
        'hw2_value': v_hw2_round,
        'hw2_policy': p_hw2,
```

```
        'hw3_value': v_hw3_round,
        'hw3_policy': p_hw3
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

2. `index.html` (前端介面)

加入了兩個區塊來分別顯示 HW1-2 和 HW1-3 的對比。

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>RL Grid World: Value Iteration</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>Grid World: Reinforcement Learning</h1>

    <div class="controls">
        <label>Enter a number between 5 and 9:</label>
        <input type="number" id="gridSize" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
    </div>

    <h2 id="gridTitle" style="display: none;"></h2>
    <div id="instruction"></div>

    <div id="gridContainer"></div>

    <button id="calculateBtn" onclick="submitToFlask()">計算策略與價值 (Submit to Flask)</button>

    <div id="results" style="display: none;">
        <hr>
        <h2 style="color: #555;">HW1-2: Policy Evaluation (Random Policy)</h2>
        <div class="result-container">
            <div class="matrix-wrapper">
                <h3>Value Function V(s)</h3>
                <div id="hw2ValueGrid"></div>
            </div>
            <div class="matrix-wrapper">
                <h3>Policy Matrix</h3>
                <div id="hw2PolicyGrid"></div>
            </div>
        </div>

        <hr>
        <h2 style="color: #b30000;">HW1-3: Value Iteration (Optimal Policy)</h2>
        <div class="result-container">
            <div class="matrix-wrapper">
                <h3>Optimal Value V*(s)</h3>
                <div id="hw3ValueGrid"></div>
            </div>
            <div class="matrix-wrapper">
                <h3>Optimal Policy</h3>
                <div id="hw3PolicyGrid"></div>
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

3. `script.js` (前端邏輯與 AJAX)

負責將地圖資料打給 `app.py`，並把四個矩陣畫出來。

```
let n = 0;
let clickState = 0;
let obstaclesLeft = 0;
let mapData = { size: 0, start: null, end: null, obstacles: [] };

window.onload = function() { generateGrid(); };

function generateGrid() {
    const inputVal = document.getElementById('gridSize').value;
    n = parseInt(inputVal);
    if (n < 5 || n > 9) return alert("Please enter a number between 5 and 9.");

    clickState = 0;
    obstaclesLeft = n - 2;
    mapData = { size: n, start: null, end: null, obstacles: [] };

    document.getElementById('gridTitle').style.display = 'block';
    document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
    document.getElementById('calculateBtn').style.display = 'none';
```

```
    document.getElementById('results').style.display = 'none';
    updateInstruction();

    const container = document.getElementById('gridContainer');
    container.innerHTML = '';

    let cellNumber = 1;
    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';
            cellDiv.innerText = cellNumber++;
            cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}

function handleCellClick(cellElement, i, j) {
    if (cellElement.classList.contains('cell-start') || cellElement.classList.contains('cell-end') || cellElement.classList.contains('cel

    if (clickState === 0) {
        cellElement.classList.add('cell-start');
        mapData.start = [i, j];
        clickState = 1;
    } else if (clickState === 1) {
        cellElement.classList.add('cell-end');
        mapData.end = [i, j];
        clickState = 2;
    } else if (clickState === 2) {
        cellElement.classList.add('cell-obstacle');
        mapData.obstacles.push([i, j]);
        obstaclesLeft--;
        if (obstaclesLeft === 0) {
            clickState = 3;
            document.getElementById('calculateBtn').style.display = 'block';
        }
    }
    updateInstruction();
}

function updateInstruction() {
    const inst = document.getElementById('instruction');
    if (clickState === 0) { inst.innerText = "1. Click to set START (Green)."; inst.style.color = "green"; }
    else if (clickState === 1) { inst.innerText = "2. Click to set END (Red)."; inst.style.color = "red"; }
    else if (clickState === 2) { inst.innerText = `3. Click to set Obstacles (Grey). ${obstaclesLeft} left.`; inst.style.color = "grey";
    else if (clickState === 3) { inst.innerText = "Setup complete! Ready to calculate."; inst.style.color = "blue"; }
}

// 發送請求給 Flask
async function submitToFlask() {
    try {
        const response = await fetch('http://127.0.0.1:5000/calculate', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(mapData)
        });

        const data = await response.json();

        document.getElementById('results').style.display = 'block';

        // 畫出四個矩陣來對比
        renderResultGrid('hw2ValueGrid', data.hw2_value, 'value');
        renderResultGrid('hw2PolicyGrid', data.hw2_policy, 'policy');
        renderResultGrid('hw3ValueGrid', data.hw3_value, 'value');
        renderResultGrid('hw3PolicyGrid', data.hw3_policy, 'policy');

    } catch (error) {
        console.error("Error:", error);
        alert("無法連接到 Flask，請確保 app.py 正在執行中 (http://127.0.0.1:5000) !");
    }
}

function renderResultGrid(containerId, matrixData, type) {
    const container = document.getElementById(containerId);
    container.innerHTML = '';
    const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';

            const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);
```

```
            if (isEnd) {
                cellDiv.classList.add('cell-end');
                cellDiv.innerText = 'END';
            } else if (isObstacle) {
                cellDiv.classList.add('cell-obstacle');
            } else {
                if (type === 'value') cellDiv.innerText = matrixData[i][j];
                else if (type === 'policy') {
                    const arrowStr = matrixData[i][j].map(a => arrows[a]).join('');
                    cellDiv.innerHTML = `<span class="arrow">${arrowStr}</span>`;
                }
            }
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}
```

4. `style.css` (排版不變，稍作微調)

```
body { font-family: Arial, sans-serif; margin: 40px; }
.controls { margin-bottom: 20px; }
#gridContainer, .matrix-wrapper > div { display: inline-block; border: 2px solid #000; background-color: #fff; }
.grid-row { display: flex; }
.grid-cell { width: 60px; height: 60px; border: 1px solid #333; display: flex; align-items: center; justify-content: center; cursor: poin
.cell-start { background-color: green; color: white; font-weight: bold; }
.cell-end { background-color: red; color: white; font-weight: bold; }
.cell-obstacle { background-color: grey; color: black; }
#instruction { font-size: 18px; font-weight: bold; margin-top: 10px; margin-bottom: 20px;}
#calculateBtn { padding: 10px 20px; font-size: 16px; background-color: #007BFF; color: white; border: none; cursor: pointer; display: non
#calculateBtn:hover { background-color: #0056b3; }
.result-container { display: flex; gap: 40px; margin-top: 15px; margin-bottom: 40px; flex-wrap: wrap; }
.matrix-wrapper { display: flex; flex-direction: column; align-items: center; }
.matrix-wrapper h3 { margin-bottom: 10px; }
.arrow { font-size: 20px; font-weight: bold; line-height: 1; }
hr { border: 0; height: 1px; background: #ccc; margin: 30px 0; }
```

🚀 如何執行

1. 把這四個檔案放在同一個資料夾。
2. 打開終端機，執行 `python app.py`。
3. **直接雙擊打開** `index.html` (或者用 VSCode 的 Live Server 打開)。
4. 點擊完格子後，按下計算按鈕。你會看到下方同時出現了**隨機政策**和**最佳政策 (Value Iteration)** 的強烈對比，這完全命中了 HW1-3 的評分要求！

You

這是hw 1-3 希望呈現的畫面

Gemini

太棒了！這張圖非常清楚地展示了你想要的效果：**不僅要顯示每個格子的最佳行動，還要把從「起點」到「終點」的實際最佳路徑（Optimal Path）用綠色高亮標示出來，並且非路徑上的箭頭要變成灰色。**

為了達成這個視覺效果，我們需要做三件事：

1. **後端 (`app.py`)**：算完最佳政策後，要多寫一個函數，從「起點」順著箭頭一路走到「終點」，把這條路徑的座標記錄下來傳給前端。
2. **前端 (`script.js`)**：在畫 Policy Matrix 的時候，檢查格子是不是在這條路徑上。如果是，就塗成綠色、箭頭變黑；如果不是，箭頭變灰。並加上 START / END 的文字。
3. **樣式 (`style.css`)**：加上對應的綠色背景和灰色箭頭的 CSS。

以下是為你修改好的三個檔案程式碼，你可以直接覆蓋原本的檔案：

1. 更新 `app.py` (加入路徑追蹤邏輯)

請將 `app.py` 完全替換成以下程式碼。我在裡面加入了一個 `trace_optimal_path` 函數。

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

ACTIONS = {'up': (-1, 0), 'down': (1, 0), 'left': (0, -1), 'right': (0, 1)}

def get_next_state(r, c, action_name, grid_size, obstacles):
    dr, dc = ACTIONS[action_name]
    next_r, next_c = r + dr, c + dc
    if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
        return r, c
    return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0):
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
```

```
            if [r, c] == end or [r, c] in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            for a in ACTIONS:
                nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                val = -1 + gamma * V[nr][nc]

                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [a]
                elif abs(val - max_val) <= 1e-6:
                    best_actions.append(a)

            policy[r][c] = best_actions
    return policy

# 追蹤最佳路徑的函數
def trace_optimal_path(policy, start, end):
    path = []
    current = tuple(start)
    visited = set() # 防止無窮迴圈

    while current != tuple(end):
        if current in visited:
            break
        visited.add(current)
        path.append(list(current))

        r, c = current
        actions = policy[r][c]
        if not actions:
            break

        # 只要順著第一個最佳行動走就好
        best_action = actions[0]
        dr, dc = ACTIONS[best_action]
        current = (r + dr, c + dc)

    path.append(list(end))
    return path

def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue
                v_expected = 0
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    v_expected += 0.25 * (-1 + gamma * V[nr][nc])
                new_V[r][c] = v_expected
                delta = max(delta, abs(v_expected - V[r][c]))
        V = new_V
        if delta < theta: break
    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue
                max_v = -float('inf')
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    v = -1 + gamma * V[nr][nc]
                    if v > max_v: max_v = v
                new_V[r][c] = max_v
                delta = max(delta, abs(max_v - V[r][c]))
        V = new_V
        if delta < theta: break
    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    size = data['size']
    start = data['start']
    end = data['end']
```

```
    obstacles = data['obstacles']

    v_hw2, p_hw2 = policy_evaluation(size, end, obstacles)
    v_hw3, p_hw3 = value_iteration(size, end, obstacles)

    # 計算最佳政策下的路徑
    optimal_path_hw3 = trace_optimal_path(p_hw3, start, end)

    v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2]
    v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3]

    return jsonify({
        'hw2_value': v_hw2_round,
        'hw2_policy': p_hw2,
        'hw3_value': v_hw3_round,
        'hw3_policy': p_hw3,
        'hw3_path': optimal_path_hw3  # 把路徑傳給前端
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

2. 更新 `script.js` (加入路徑渲染與灰色箭頭)

這裡我們修改了 `renderResultGrid` 函數，讓它可以接收路徑參數，並畫出與你圖片一模一樣的效果。

```
let n = 0;
let clickState = 0;
let obstaclesLeft = 0;
let mapData = { size: 0, start: null, end: null, obstacles: [] };

window.onload = function() { generateGrid(); };

function generateGrid() {
    const inputVal = document.getElementById('gridSize').value;
    n = parseInt(inputVal);
    if (n < 5 || n > 9) return alert("Please enter a number between 5 and 9.");

    clickState = 0;
    obstaclesLeft = n - 2;
    mapData = { size: n, start: null, end: null, obstacles: [] };

    document.getElementById('gridTitle').style.display = 'block';
    document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
    document.getElementById('calculateBtn').style.display = 'none';
    document.getElementById('results').style.display = 'none';
    updateInstruction();

    const container = document.getElementById('gridContainer');
    container.innerHTML = '';

    let cellNumber = 1;
    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';
            cellDiv.innerText = cellNumber++;
            cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}

function handleCellClick(cellElement, i, j) {
    if (cellElement.classList.contains('cell-start') || cellElement.classList.contains('cell-end') || cellElement.classList.contains('cel

    if (clickState === 0) {
        cellElement.classList.add('cell-start');
        mapData.start = [i, j];
        clickState = 1;
    } else if (clickState === 1) {
        cellElement.classList.add('cell-end');
        mapData.end = [i, j];
        clickState = 2;
    } else if (clickState === 2) {
        cellElement.classList.add('cell-obstacle');
        mapData.obstacles.push([i, j]);
        obstaclesLeft--;
        if (obstaclesLeft === 0) {
            clickState = 3;
            document.getElementById('calculateBtn').style.display = 'block';
        }
    }
    updateInstruction();
}

function updateInstruction() {
    const inst = document.getElementById('instruction');
```

```
    if (clickState === 0) { inst.innerText = "1. Click to set START (Green)."; inst.style.color = "green"; }
    else if (clickState === 1) { inst.innerText = "2. Click to set END (Red)."; inst.style.color = "red"; }
    else if (clickState === 2) { inst.innerText = `3. Click to set Obstacles (Grey). ${obstaclesLeft} left.`; inst.style.color = "grey";
    else if (clickState === 3) { inst.innerText = "Setup complete! Ready to calculate."; inst.style.color = "blue"; }
}

async function submitToFlask() {
    try {
        const response = await fetch('http://127.0.0.1:5000/calculate', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(mapData)
        });

        const data = await response.json();

        document.getElementById('results').style.display = 'block';

        // 畫出結果，HW3 額外傳入 optimal_path
        renderResultGrid('hw2ValueGrid', data.hw2_value, 'value');
        renderResultGrid('hw2PolicyGrid', data.hw2_policy, 'policy');
        renderResultGrid('hw3ValueGrid', data.hw3_value, 'value');
        renderResultGrid('hw3PolicyGrid', data.hw3_policy, 'policy', data.hw3_path);

    } catch (error) {
        console.error("Error:", error);
        alert("無法連接到 Flask，請確保 app.py 正在執行中 !");
    }
}

// 加入 optimalPath 參數，用來高亮最佳路徑
function renderResultGrid(containerId, matrixData, type, optimalPath = null) {
    const container = document.getElementById(containerId);
    container.innerHTML = '';

    // 使用比較粗的箭頭符號
    const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';

            const isStart = (mapData.start[0] === i && mapData.start[1] === j);
            const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);
            const isPath = optimalPath && optimalPath.some(p => p[0] === i && p[1] === j);

            if (isObstacle) {
                cellDiv.classList.add('cell-obstacle');
            } else {
                // 如果是在最佳路徑上，加上高亮 class
                if (isPath) {
                    cellDiv.classList.add('cell-path');
                }

                if (type === 'value') {
                    if (isEnd) cellDiv.innerText = 'END';
                    else cellDiv.innerText = matrixData[i][j];
                } else if (type === 'policy') {
                    let content = '';

                    // 根據圖片，START 跟 END 要有小字
                    if (isStart) content += '<div class="small-text">START</div>';

                    if (!isEnd) {
                        const actionList = matrixData[i][j];
                        // 沒有 action 時 (例如被障礙物包圍)，不顯示箭頭
                        if (actionList.length > 0) {
                            const arrowStr = actionList.map(a => arrows[a]).join('');
                            const arrowClass = isPath ? 'arrow-path' : 'arrow-normal';
                            content += `<div class="arrow ${arrowClass}">${arrowStr}</div>`;
                        }
                    } else {
                        content += '<div class="small-text">END</div>';
                    }

                    cellDiv.innerHTML = content;
                }
            }
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}
```

3. 更新 `style.css` (加入路徑顏色)

把這幾段新的 CSS 加到你原本的 `style.css` 裡面（或是直接貼到底部覆蓋相關設定）：

```
/* ... 保留之前的 CSS ... */

/* 新增：最佳路徑的高亮背景 (與你圖片一樣的綠色) */
.cell-path {
    background-color: #71be58; /* 綠色 */
    border: 1px solid #000;
}

/* 箭頭樣式調整 */
.arrow {
    font-size: 26px; /* 把箭頭調大一點 */
    font-weight: bold;
    line-height: 1;
}

/* 在路徑上的箭頭是黑色的 */
.arrow-path {
    color: #000;
}

/* 不在路徑上的箭頭是灰色的 */
.arrow-normal {
    color: #888;
}

/* START 和 END 的小字體 */
.small-text {
    font-size: 10px;
    font-weight: bold;
    color: #000;
    margin-bottom: -5px; /* 調整讓箭頭跟字靠近一點 */
}

/* 為了讓 START 裡面同時塞文字和箭頭，把 grid-cell 改成 flex-col */
.grid-cell {
    width: 60px;
    height: 60px;
    border: 1px solid #333;
    display: flex;
    flex-direction: column; /* 改為垂直排列 */
    align-items: center;
    justify-content: center;
    cursor: pointer;
    box-sizing: border-box;
    font-size: 16px;
    user-select: none;
    text-align: center;
}
```

完成這三步後，重新啟動你的 Flask，然後重整網頁。當你按下計算按鈕後，**HW1-3 的 Policy Matrix 就會畫出一條從 START 完美連到 END 的綠色大道，旁邊的路徑則是灰色的箭頭**，完全重現你上傳的參考圖！

試試看，如果有哪個顏色或箭頭大小想要微調，隨時跟我說！

You

為什麼會有一格內兩個箭頭的情況

Gemini

這是一個非常棒的問題！你觀察得非常仔細。

在強化學習的網格世界中，當一個格子裡出現兩個（或多個）箭頭時，這代表**「這幾個方向都是『最佳』選擇，它們一樣好」**（也就是發生了平手，Tie）。

為什麼會「一樣好」？

演算法（價值迭代 Value Iteration）在決定箭頭方向時，會計算往上、下、左、右走，哪一個方向能得到「最大的未來總價值（或是說最快抵達終點）」。

如果演算法發現，不管你選哪個方向，**走到終點的總步數（或總代價）是一模一樣的**，它就無法說哪一個絕對比較好，於是就會把這幾個最好的方向都顯示出來。

🔍 讓我們用你圖片中的例子來驗證：

看看圖片左下角算上來第二排、最左邊的那個格子（裡面有 ↓ 和 → 的灰色箭頭）。我們來算一下它走到 END 的步數：

1. **如果選擇往下 (↓)：** 往下 1 步 $\rightarrow$ 往右 4 步 $\rightarrow$ 抵達 END。**總共需要 5 步**。
2. **如果選擇往右 (→)：** 往右 1 步 $\rightarrow$ 往下 1 步 $\rightarrow$ 往右 3 步 $\rightarrow$ 抵達 END。**總共需要 5 步**。

你看！因為兩條路徑花費的步數完全相同，預期價值 $V(s)$ 算出來一模一樣，所以演算法很誠實地告訴你：「往下或往右走都可以喔，都是最快的捷徑！」

💻 程式碼裡是怎麼做到的？

還記得我們在 `app.py` 裡面寫的 `derive_policy` 邏輯嗎？這正是那段程式碼發揮作用的地方：

```
# 如果發現這個方向的價值 (val) 大於目前的最高分 (max_val)
if val > max_val + 1e-6:
    max_val = val
    best_actions = [a]  # 把最佳行動替換成這個新的

# 如果發現這個方向的價值跟最高分「一樣高」(差距極小)
elif abs(val - max_val) <= 1e-6:
    best_actions.append(a)  # 把這個方向也加進最佳行動清單裡
```

正是因為有那個 `elif` 判斷式，程式才會把所有「同樣好」的動作都收集起來，變成你在畫面上看到的多箭頭！

這樣的解釋有解答你的疑惑嗎？還是說你希望程式永遠只顯示「唯一」一個箭頭（比如平手時優先選往下）呢？

You

我希望最終答案只有一個箭頭 用來表示它最終走哪條路

Gemini

明白你的需求了！如果畫面上充滿雙箭頭會讓人覺得有點眼花撩亂，強制每個格子「只選一個最優解（打破平手）」，確實能讓畫面看起來更簡潔，也能更明確地看出唯一的最佳路徑。

要達成這個效果非常簡單，我們只需要修改 `app.py` 裡的 `derive_policy` 函數。

我們把原本「遇到平手就把方向加進清單」的邏輯刪除，改成**「只有遇到『嚴格大於』目前最高分的方向時，才替換箭頭」**。這樣一來，程式就會根據我們定義動作的順序（上下左右），保留它第一個找到的最佳方向。

🛠️ 修改 `app.py`

請打開你的 `app.py`，找到 `derive_policy` 這個函數，把它替換成下面這段程式碼：

```
def derive_policy(V, grid_size, end, obstacles, gamma=1.0):
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
            if [r, c] == end or [r, c] in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            for a in ACTIONS:
                nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                val = -1 + gamma * V[nr][nc]

                # 修改這裡：只要嚴格大於目前最大值才更新，遇到平手就不理它！
                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [a]  # 永遠只保留一個方向

            policy[r][c] = best_actions
    return policy
```

修改重點說明： 我把原本的 `elif abs(val - max_val) <= 1e-6: best_actions.append(a)` 這兩行刪掉了。這代表當程式發現「往下」和「往右」一樣好時，它只會記住它先算到的那個方向，不再把另一個方向加進去。

存檔後，重新執行 `python app.py` 並重整網頁。你會發現現在**每個格子裡都只剩下一個乾淨俐落的箭頭**了！

你覺得這樣的呈現方式符合你的期待嗎？需要我幫你調整字體大小或是綠色路徑的顏色嗎？

You

我把你給的 function 換過去後 看起來 app.py 接不到 req 了

from flask import Flask, request, jsonify from flask_cors import CORS

app = Flask(**name**) CORS(app)

ACTIONS = {'up': (-1, 0), 'down': (1, 0), 'left': (0, -1), 'right': (0, 1)}

def get_next_state(r, c, action_name, grid_size, obstacles): dr, dc = ACTIONS[action_name] next_r, next_c = r + dr, c + dc if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles: return r, c return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0): policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue max_val = -float('inf') best_actions = [] for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) val = -1 + gamma * V[nr][nc] # 修改這裡：只要嚴格大於目前最大值才更新，遇到平手就不理它！ if val > max_val + 1e-6: max_val = val best_actions = [a] # 永遠只保留一個方向 policy[r][c] = best_actions return policy

追蹤最佳路徑的函數

def trace_optimal_path(policy, start, end): path = [] current = tuple(start) visited = set() # 防止無窮迴圈 while current != tuple(end): if current in visited: break visited.add(current) path.append(list(current)) r, c = current actions = policy[r][c] if not actions: break

只要順著第一個最佳行動走就好 best_action = actions[0] dr, dc = ACTIONS[best_action] current = (r + dr, c + dc) path.append(list(end)) return path

def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0): V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] while True: delta = 0 new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue v_expected = 0 for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) v_expected += 0.25 * (-1 + gamma * V[nr][nc]) new_V[r][c] = v_expected delta = max(delta, abs(v_expected - V[r][c])) V = new_V if delta < theta: break policy = derive_policy(V, grid_size, end, obstacles, gamma) return V, policy

def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0): V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] while True: delta = 0 new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue max_v = -float('inf') for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) v = -1 + gamma * V[nr][nc] if v > max_v: max_v = v new_V[r][c] = max_v delta = max(delta, abs(max_v - V[r][c])) V = new_V if delta < theta: break policy = derive_policy(V, grid_size, end, obstacles, gamma) return V, policy

@app.route('/calculate', methods=['POST']) def calculate(): data = request.json size = data['size'] start = data['start'] end = data['end'] obstacles = data['obstacles'] v_hw2, p_hw2 = policy_evaluation(size, end, obstacles) v_hw3, p_hw3 = value_iteration(size, end, obstacles) # 計算最佳政策下的路徑 optimal_path_hw3 = trace_optimal_path(p_hw3, start, end) v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2] v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3] return jsonify({ 'hw2_value': v_hw2_round, 'hw2_policy': p_hw2, 'hw3_value': v_hw3_round, 'hw3_policy': p_hw3, 'hw3_path': optimal_path_hw3 # 把路徑傳給前端 })

if **name** == '**main**': app.run(debug=True, port=5000)

我把你給的 function 換過去後 看起來 app.py 接不到 req 了

from flask import Flask, request, jsonify from flask_cors import CORS

app = Flask(**name**) CORS(app)

ACTIONS = {'up': (-1, 0), 'down': (1, 0), 'left': (0, -1), 'right': (0, 1)}

def get_next_state(r, c, action_name, grid_size, obstacles): dr, dc = ACTIONS[action_name] next_r, next_c = r + dr, c + dc if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles: return r, c return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0): policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue max_val = -float('inf') best_actions = [] for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) val = -1 + gamma * V[nr][nc] # 修改這裡：只要嚴格大於目前最大值才更新，遇到平手就不理它！ if val > max_val + 1e-6: max_val = val best_actions = [a] # 永遠只保留一個方向 policy[r][c] = best_actions return policy

追蹤最佳路徑的函數

def trace_optimal_path(policy, start, end): path = [] current = tuple(start) visited = set() # 防止無窮迴圈 while current != tuple(end): if current in visited: break visited.add(current) path.append(list(current)) r, c = current actions = policy[r][c] if not actions: break
只要順著第一個最佳行動走就好 best_action = actions[0] dr, dc = ACTIONS[best_action] current = (r + dr, c + dc) path.append(list(end)) return path

def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0): V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] while True: delta = 0 new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue v_expected = 0 for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) v_expected += 0.25 * (-1 + gamma * V[nr][nc]) new_V[r][c] = v_expected delta = max(delta, abs(v_expected - V[r][c])) V = new_V if delta < theta: break policy = derive_policy(V, grid_size, end, obstacles, gamma) return V, policy

def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0): V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] while True: delta = 0 new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue max_v = -float('inf') for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) v = -1 + gamma * V[nr][nc] if v > max_v: max_v = v new_V[r][c] = max_v delta = max(delta, abs(max_v - V[r][c])) V = new_V if delta < theta: break policy = derive_policy(V, grid_size, end, obstacles, gamma) return V, policy

@app.route('/calculate', methods=['POST']) def calculate(): data = request.json size = data['size'] start = data['start'] end = data['end'] obstacles = data['obstacles'] v_hw2, p_hw2 = policy_evaluation(size, end, obstacles) v_hw3, p_hw3 = value_iteration(size, end, obstacles) # 計算最佳政策下的路徑 optimal_path_hw3 = trace_optimal_path(p_hw3, start, end) v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2] v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3] return jsonify({ 'hw2_value': v_hw2_round, 'hw2_policy': p_hw2, 'hw3_value': v_hw3_round, 'hw3_policy': p_hw3, 'hw3_path': optimal_path_hw3 # 把路徑傳給前端 })

if **name** == '**main**': app.run(debug=True, port=5000)

Gemini

這通常是因為一些編碼或是縮排的隱藏字元導致 Python 腳本無法正常執行或卡住，又或者是 Flask 伺服器沒有正確被重啟。

我們一步步來排除這個問題。

步驟 1：檢查 Python 檔案是否有隱藏字元

你貼上的這段程式碼中，很多行首有全形空格或是不正常的空白字元（這通常發生在從網頁或 PDF 複製貼上時）。這會導致 Python 解析錯誤。

我已經幫你重新整理並**去除了所有不正常的空白字元**，請將你 `app.py` 的內容**完全清空**，然後直接複製貼上這段純淨的程式碼：

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

ACTIONS = {'up': (-1, 0), 'down': (1, 0), 'left': (0, -1), 'right': (0, 1)}

def get_next_state(r, c, action_name, grid_size, obstacles):
    dr, dc = ACTIONS[action_name]
    next_r, next_c = r + dr, c + dc
    if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles:
```

```
        return r, c
    return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0):
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
            if [r, c] == end or [r, c] in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            for a in ACTIONS:
                nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                val = -1 + gamma * V[nr][nc]

                # 只要嚴格大於目前最大值才更新，遇到平手就不理它！
                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [a]  # 永遠只保留一個方向

            policy[r][c] = best_actions
    return policy

def trace_optimal_path(policy, start, end):
    path = []
    current = tuple(start)
    visited = set()

    while current != tuple(end):
        if current in visited:
            break
        visited.add(current)
        path.append(list(current))

        r, c = current
        actions = policy[r][c]
        if not actions:
            break

        best_action = actions[0]
        dr, dc = ACTIONS[best_action]
        current = (r + dr, c + dc)

    path.append(list(end))
    return path

def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue
                v_expected = 0
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    v_expected += 0.25 * (-1 + gamma * V[nr][nc])
                new_V[r][c] = v_expected
                delta = max(delta, abs(v_expected - V[r][c]))
        V = new_V
        if delta < theta: break
    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue
                max_v = -float('inf')
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    v = -1 + gamma * V[nr][nc]
                    if v > max_v: max_v = v
                new_V[r][c] = max_v
                delta = max(delta, abs(max_v - V[r][c]))
        V = new_V
        if delta < theta: break
    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
```

```
    size = data['size']
    start = data['start']
    end = data['end']
    obstacles = data['obstacles']

    v_hw2, p_hw2 = policy_evaluation(size, end, obstacles)
    v_hw3, p_hw3 = value_iteration(size, end, obstacles)

    optimal_path_hw3 = trace_optimal_path(p_hw3, start, end)

    v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2]
    v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3]

    return jsonify({
        'hw2_value': v_hw2_round,
        'hw2_policy': p_hw2,
        'hw3_value': v_hw3_round,
        'hw3_policy': p_hw3,
        'hw3_path': optimal_path_hw3
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

步驟 2：確認伺服器是否有確實重啟

1. 到你執行 Flask 的終端機 (Terminal / Command Prompt)。
2. 按下 `Ctrl + C` 終止目前的伺服器。
3. 再次執行指令：`python app.py`。
4. 確保畫面上出現類似 `* Running on http://127.0.0.1:5000` 且沒有報錯。

步驟 3：在瀏覽器中強制作廢快取 (Hard Reload)

有時候前端 `script.js` 被瀏覽器快取了，導致發送請求的行為還是舊的。

1. 回到你的網頁 (index.html)。
2. 按下 `Ctrl + F5` (Windows) 或 `Cmd + Shift + R` (Mac) 進行強制重新整理。
3. 按 `F12` 打開開發者工具，切換到 `Console` 標籤。
4. 再次點擊網格設定並按下「計算策略與價值」。

觀察 `Console` 裡面有沒有紅色的錯誤訊息？或是終端機 (跑 Flask 的視窗) 裡面有沒有印出錯誤的 Log？把這些訊息告訴我，我們就能馬上抓到蟲！

You

我讓它在 vercel 上跑, 我有辦法在 vercel 上跑 app.py 嗎

Gemini

可以的！Vercel 確實可以跑 Python (Flask)，但它的運作邏輯跟傳統伺服器不一樣。

Vercel 是一個主打「無伺服器 (Serverless)」的平台。它不會像你的電腦一樣在背景持續執行 `python app.py` 並監聽 5000 port。相反地，它會把你的 Python 程式碼變成一個**「無伺服器函數 (Serverless Function)」**，只有在前端發送請求時，它才會瞬間啟動運算，算完就關掉。

要讓你的 Flask 應用程式在 Vercel 上順利運作，我們需要稍微調整一下**資料夾結構**和**程式碼**。請跟著我做這 5 個步驟：

步驟 1：調整資料夾結構

Vercel 規定所有的後端 API 都必須放在一個名為 `api` 的資料夾裡面。請把你的 `app.py` 移進去，並改名為 `index.py`。

你的資料夾結構最後要長得像這樣：

```
你的專案資料夾/
├── index.html
├── style.css
├── script.js
├── requirements.txt   <-- 新增這個檔案
├── vercel.json        <-- 新增這個檔案
└── api/
    └── index.py       <-- 這就是原本的 app.py
```

步驟 2：建立 `requirements.txt`

Vercel 需要知道你的 Python 裝了哪些套件。在專案根目錄建立一個 `requirements.txt`，裡面寫入：

```
Flask==3.0.0
Flask-Cors==4.0.0
```

步驟 3：建立 `vercel.json` (設定路由)

這是一個設定檔，用來告訴 Vercel：「當前端呼叫 `/calculate` 的時候，請把請求轉交給 `api/index.py` 處理」。 在專案根目錄建立 `vercel.json`：

```
{
  "rewrites": [
    {
```

```
      "source": "/calculate",
      "destination": "/api/index"
    }
  ]
}
```

步驟 4：微調 `api/index.py` (原本的 app.py)

把我們剛剛那份完美的 `app.py` 貼進 `api/index.py` 裡，但**請把最後面的這兩行刪掉**：

```
# --- 請把這兩行刪除，因為 Vercel 不需要 app.run() ---
# if __name__ == '__main__':
#     app.run(debug=True, port=5000)
```

Vercel 在底層會自動接管 `app` 這個 Flask 實例，所以不需要我們手動 `app.run()`。

步驟 5：修改前端 `script.js` 的 API 網址

因為現在前後端都部署在同一個 Vercel 網域下了，我們不能再向 `http://127.0.0.1:5000` (你的本機) 發送請求。

請打開 `script.js`，找到 `submitToFlask` 函數，把 `fetch` 的網址改成**相對路徑**：

```
// 原本是： const response = await fetch('http://127.0.0.1:5000/calculate', ...
// 改成這樣：
const response = await fetch('/calculate', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(mapData)
});
```

🚀 重新部署到 Vercel

完成這 5 個步驟後，把所有變更 `git push` 到你的 GitHub，Vercel 就會自動幫你重新部署。

部署完成後，打開 Vercel 給你的專屬網址，點擊「計算策略與價值」，你的 Flask 後端就會以 Serverless 的形式完美在雲端幫你算出結果了！

試著推上去看看，如果在 Vercel 的部署日誌 (Logs) 裡有看到任何紅字報錯，隨時複製貼上給我看！

You

我改完 push 後測試，開 f12 寫 fail to fetch, 段落是 const response = await fetch('/calculate', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(mapData) });

Gemini

出現 `Failed to fetch` 通常代表**瀏覽器把這個請求擋下來了**。在 Vercel 上遇到這個問題，99% 是因為 Vercel 後端發生了錯誤（例如 404 找不到路徑，或是 500 程式碼當機），導致 Vercel 回傳了一個錯誤網頁。因為這個錯誤網頁沒有包含 CORS 標頭，瀏覽器就會顯示 `Failed to fetch`。

要完美解決 Flask 與 Vercel 的路由問題，我們需要讓 Vercel 的網址路徑跟 Flask 內部的 `@app.route` **完全一致**。

請跟著我修改這三個檔案，把路徑統一加上 `/api`：

1. 修改 `api/index.py` (Flask 路由)

請打開 `api/index.py`，滑到最下面找到 `@app.route` 的地方，把原本的 `/calculate` 改成 `/api/calculate`：

```
# ... 前面演算法的程式碼都不用動 ...

# 這裡把原本的 '/calculate' 改成 '/api/calculate'
@app.route('/api/calculate', methods=['POST'])
def calculate():
    data = request.json
    size = data['size']
    start = data['start']
    end = data['end']
    obstacles = data['obstacles']

    v_hw2, p_hw2 = policy_evaluation(size, end, obstacles)
    v_hw3, p_hw3 = value_iteration(size, end, obstacles)
    optimal_path_hw3 = trace_optimal_path(p_hw3, start, end)

    v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2]
    v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3]

    return jsonify({
        'hw2_value': v_hw2_round,
        'hw2_policy': p_hw2,
        'hw3_value': v_hw3_round,
        'hw3_policy': p_hw3,
        'hw3_path': optimal_path_hw3
    })

# 注意：Vercel 不需要 app.run()，這兩行可以刪掉或註解掉
# if __name__ == '__main__':
#     app.run(debug=True, port=5000)
```

---

2. 修改 `vercel.json` (Vercel 路由設定)

我們設定一個「萬用路由」，讓 Vercel 把所有開頭是 `/api/` 的請求，都乖乖交給 Flask (`api/index.py`) 來處理。請將 `vercel.json` 改成這樣：

```
{
  "rewrites": [
    {
      "source": "/api/(.*)",
      "destination": "/api/index"
    }
  ]
}
```

---

3. 修改 `script.js` (前端發送請求)

最後，讓前端的 JavaScript 把請求打到剛剛設定好的 `/api/calculate` 網址。

找到 `submitToFlask()` 函數：

```
// ...
async function submitToFlask() {
    try {
        // 把這裡的 '/calculate' 改成 '/api/calculate'
        const response = await fetch('/api/calculate', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(mapData)
        });

        // ... 如果 response 沒有 OK，印出錯誤狀態碼方便除錯 ...
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
// ... 下面的程式碼保持不變 ...
```

🔗 進階除錯技巧 (如果 push 之後還是失敗)：

如果你把這三個檔案修改並 push 到 Github，Vercel 重新部署後，點擊按鈕**還是**出現 `Failed to fetch` 或其他錯誤，請一定要做這件事：

1. 登入 Vercel 儀表板。
2. 點進你的專案，點擊上方的 **"Logs" (日誌)** 標籤。
3. 再次在你的網頁上點擊「計算策略與價值」按鈕。
4. 這時 Vercel 的 Logs 畫面會跳出**紅色的 Python 錯誤訊息**（例如缺套件、語法錯誤等）。

只要把 Logs 裡面的紅色錯誤訊息複製貼上給我，我們就能一秒抓出蟲在哪裡！試試看吧！

You

接下來剩美觀的部分, 分成不同段落完成

原本的文字是按一下換一條, 我想換成 step1... step2... step3...

多一個 reset button, 用來重新設定 start, end, Obstacles

Gemini

這個 UI 的改進方向非常棒！將步驟全部列出來並加上高亮提示，可以讓使用者在操作前就清楚知道整個流程；而新增的 Reset 按鈕也能大大減少點錯時的挫折感。

我們照你說的，分成三個段落來完成這個美化工程：

🟢 第一段：修改 `index.html` (替換提示區塊與新增按鈕)

請打開你的 `index.html`，我們要把原本單行顯示的 `<div id="instruction"></div>` 換成一個列表，並在「計算按鈕」旁邊加上「Reset 按鈕」。

找到這段程式碼：

```
    <h2 id="gridTitle" style="display: none;"></h2>
    <div id="instruction"></div>

    <div id="gridContainer"></div>

    <button id="calculateBtn" onclick="submitToFlask()">計算策略與價值 (Submit to Flask)</button>
```

把它替換成以下內容：

```
    <h2 id="gridTitle" style="display: none;"></h2>

    <div id="instruction-steps">
        <p id="step1">Step 1: Click to set START (Green)</p>
        <p id="step2">Step 2: Click to set END (Red)</p>
        <p id="step3">Step 3: Click to set Obstacles (Grey)</p>
        <p id="step4" style="display:none;">Setup complete! Ready to calculate.</p>
    </div>
```

```
    <div id="gridContainer"></div>

    <div class="action-buttons">
        <button id="resetBtn" onclick="resetGrid()" style="display: none;">Reset Grid</button>
        <button id="calculateBtn" onclick="submitToFlask()" style="display: none;">計算策略與價值 (Calculate)</button>
    </div>
```

🟢 第二段：修改 `style.css` (加入步驟高亮與按鈕排版)

接著我們要在 CSS 裡加上步驟文字的顏色切換（未進行時是灰色，進行中是黑色加粗），以及讓 Reset 按鈕跟 Calculate 按鈕並排。

請打開 `style.css`，把跟 `#instruction` 有關的設定刪除，然後**加入以下這段新的 CSS**：

```
/* --- 步驟提示區塊樣式 --- */
#instruction-steps {
    margin: 15px 0;
}

#instruction-steps p {
    color: #bbb; /* 預設為淡灰色，表示未進行或已完成 */
    margin: 5px 0;
    font-size: 16px;
    transition: color 0.3s ease; /* 讓顏色切換有平滑的動畫感 */
}

/* 正在進行中的步驟，會套用這個 class 變黑變粗 */
#instruction-steps p.active-step {
    color: #000;
    font-weight: bold;
    font-size: 18px;
}

#step4 {
    color: #007BFF; /* 完成提示用藍色 */
    font-weight: bold;
    font-size: 18px;
}

/* --- 按鈕排版 --- */
.action-buttons {
    display: flex;
    gap: 15px; /* 讓兩個按鈕中間有空隙 */
    margin-top: 20px;
}

#resetBtn {
    padding: 10px 20px;
    font-size: 16px;
    background-color: #6c757d; /* 灰色按鈕 */
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 4px;
}

#resetBtn:hover {
    background-color: #5a6268;
}
```

🟢 第三段：修改 `script.js` (更新提示邏輯與 Reset 功能)

最後一步，我們要讓 JavaScript 知道什麼時候該把哪個步驟「點亮」，以及實作 `resetGrid()` 函數。

請打開 `script.js`，進行以下兩個修改：

1. 修改 `generateGrid()` 函數，讓網格生成時顯示 Reset 按鈕： 在 `generateGrid()` 函數裡，找到隱藏 Calculate 按鈕的地方，並加上一行顯示 Reset 按鈕：

```
    // ... 前面的程式碼 ...
    document.getElementById('calculateBtn').style.display = 'none';
    document.getElementById('resetBtn').style.display = 'block'; // 新增這行：顯示 Reset 按鈕
    document.getElementById('results').style.display = 'none';
    // ...
```

2. 完全替換掉原本的 `updateInstruction()` 函數，並加上 `resetGrid()`： 找到原本的 `updateInstruction()`，將它替換成以下內容：

```
function updateInstruction() {
    // 1. 先把所有步驟的「高亮狀態」移除
    document.getElementById('step1').classList.remove('active-step');
    document.getElementById('step2').classList.remove('active-step');
    document.getElementById('step3').classList.remove('active-step');
    document.getElementById('step4').style.display = 'none';

    // 2. 根據目前的點擊狀態，點亮對應的步驟
    if (clickState === 0) {
        document.getElementById('step1').classList.add('active-step');
        document.getElementById('step3').innerText = "Step 3: Click to set Obstacles (Grey)";
```

```
    } else if (clickState === 1) {
        document.getElementById('step2').classList.add('active-step');
    } else if (clickState === 2) {
        document.getElementById('step3').classList.add('active-step');
        document.getElementById('step3').innerText = `Step 3: Click to set Obstacles (Grey). ${obstaclesLeft} left.`;
    } else if (clickState === 3) {
        // 設定完成，顯示完成文字
        document.getElementById('step4').style.display = 'block';
    }
}

// 新增的 Reset 函數
function resetGrid() {
    // 其實我們原本寫好的 generateGrid() 裡面就包含了重置所有變數和清空網格的邏輯
    // 所以直接呼叫它，就能完美達到 Reset 的效果！
    generateGrid();
}
```

這樣就大功告成了！存檔後重新整理網頁，你就會看到：

1. 畫面上列出了三個步驟，當你設定起點時，Step 1 是黑色的，設定完後 Step 1 變灰，換 Step 2 變黑。
2. 畫面下方多了一個灰色的 **"Reset Grid"** 按鈕，點下去就會瞬間把綠色、紅色、灰色的格子全部洗掉，讓你重新點擊。

看看這個效果是不是你要的美觀升級！

You

幫我做防呆，避免沒有路徑的時候還在計算

Gemini

這個防呆機制非常關鍵！你找出了強化學習網格世界裡一個很經典的邊界情況（Edge Case）：**「如果起點被死路包圍，演算法會因為永遠走不到終點，導致那一區的預期價值發散到負無限大。」**

為了解決這個問題，最聰明且節省伺服器資源的做法是：**在前端按下「計算」按鈕時，先用一個輕量級的尋路演算法（例如 BFS 廣度優先搜尋）檢查一下到底有沒有路徑。如果沒有路徑，直接跳出警告並擋下請求。**

請打開你的 `script.js` 檔案，我們分成兩個步驟來加入這個防呆功能：

步驟 1：加入路徑檢查函數 (`hasValidPath`)

請將這段新的函數加在 `script.js` 的**最下方**。它會利用 BFS (廣度優先搜尋) 快速掃描一次地圖，看看是否能從綠色走到紅色：

```
// --- 新增：使用 BFS (廣度優先搜尋) 檢查是否有路徑 ---
function hasValidPath() {
    const size = mapData.size;
    const start = mapData.start;
    const end = mapData.end;
    const obstacles = mapData.obstacles;

    // 判斷是否為障礙物的輔助函數
    const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c);

    // 建立一個紀錄是否走過的矩陣
    let visited = Array.from({ length: size }, () => Array(size).fill(false));

    // 將起點放入佇列
    let queue = [start];
    visited[start[0]][start[1]] = true;

    // 上下左右四個方向
    const dirs = [[-1, 0], [1, 0], [0, -1], [0, 1]];

    while (queue.length > 0) {
        let current = queue.shift();
        let r = current[0];
        let c = current[1];

        // 如果走到終點，代表有路徑！
        if (r === end[0] && c === end[1]) {
            return true;
        }

        // 探索四個方向
        for (let dir of dirs) {
            let nextR = r + dir[0];
            let nextC = c + dir[1];

            // 檢查是否超出邊界
            if (nextR >= 0 && nextR < size && nextC >= 0 && nextC < size) {
                // 如果不是障礙物，且還沒走過
                if (!isObstacle(nextR, nextC) && !visited[nextR][nextC]) {
                    visited[nextR][nextC] = true;
                    queue.push([nextR, nextC]);
                }
```

```
            }
        }
    }

    // 佇列都清空了還是沒找到終點，代表被死路包圍
    return false;
}
```

步驟 2：在送出請求前攔截 (`submitToFlask`)

接著，找到你原本發送 API 請求的 `submitToFlask()` 函數，在它的**最開頭**加上防呆攔截邏輯：

```
async function submitToFlask() {
    // --- 在這裡加入防呆攔截 ---
    if (!hasValidPath()) {
        alert("⚠️ 錯誤：起點和終點之間被障礙物完全阻擋，找不到可通行的路徑！請點擊 Reset Grid 重新設定。");
        return; // 直接中斷執行，不會把資料傳給 Flask 浪費運算資源
    }
    // ---------------------------

    try {
        const response = await fetch('/api/calculate', {  // 或者是你原本的 API 網址
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(mapData)
        });

        // ... 下面的程式碼維持不變 ...
```

💡 測試一下

存檔並重新整理網頁，照著你圖片上的方式，把起點 1 用障礙物 2、6、7 封死，然後按下「Calculate」。

你應該會立刻看到一個警告視窗彈出來說：**「⚠️ 錯誤：起點和終點之間被障礙物完全阻擋...」**，並且不會畫出任何錯誤的矩陣圖表。

這樣就完美解決了無效路徑導致的運算問題了！前端的防呆體驗現在非常完整。你接下來還有需要調整哪裡的細節嗎？

You

幫我讓整個畫面更美觀

Gemini

這是一個讓專案從「作業」升級成「現代化作品集」的絕佳決定！

為了讓畫面看起來更專業、現代且具有科技感，我幫你重新設計了整個視覺風格（UI/UX）。這次的升級重點包括：

1. **套用現代字體**：引入 Google Fonts 的 `Poppins` 字體，讓文字看起來更圓潤舒適。
2. **卡片式排版與陰影**：將主畫面置中，並加上柔和的陰影與圓角設計（Card Design）。
3. **色彩優化（Material Design）**：把原本刺眼的純紅、純綠，換成柔和且具質感的翠綠色、珊瑚紅與石板灰。
4. **按鈕與互動特效**：加上 hover（懸停）的浮動效果與漸變轉場。

請分別覆蓋你的 `index.html` 和 `style.css`，JavaScript 不用動！

🟢 1. 更新 `index.html` (加入外層容器與字體)

我在 `<head>` 引入了 Google 字體，並把所有內容包在一個 `<div class="app-container">` 裡面，方便置中排版。

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>RL Grid World: Value Iteration</title>
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="app-container">
        <header>
            <h1>Grid World <span class="badge">RL</span></h1>
            <p class="subtitle">Policy Evaluation & Value Iteration</p>
        </header>

        <div class="controls card">
            <label for="gridSize">Grid Size (5-9):</label>
            <input type="number" id="gridSize" min="5" max="9" value="5">
            <button class="btn-primary" onclick="generateGrid()">Generate</button>
        </div>

        <div class="setup-section card">
            <h2 id="gridTitle" style="display: none;"></h2>
```

```
            <div id="instruction-steps">
                <p id="step1">Step 1: Click to set <strong>START</strong> (Green)</p>
                <p id="step2">Step 2: Click to set <strong>END</strong> (Red)</p>
                <p id="step3">Step 3: Click to set <strong>Obstacles</strong> (Grey)</p>
                <p id="step4" style="display:none;">✨ Setup complete! Ready to calculate.</p>
            </div>

            <div class="grid-wrapper">
                <div id="gridContainer"></div>
            </div>

            <div class="action-buttons">
                <button id="resetBtn" class="btn-secondary" onclick="resetGrid()" style="display: none;">Reset Grid</button>
                <button id="calculateBtn" class="btn-success" onclick="submitToFlask()" style="display: none;">Calculate Policy</button>
            </div>
        </div>

        <div id="results" style="display: none;">

            <div class="result-card">
                <h2 class="section-title">HW1-2: Policy Evaluation <span>(Random Policy)</span></h2>
                <div class="result-container">
                    <div class="matrix-wrapper">
                        <h3>Value Function V(s)</h3>
                        <div id="hw2ValueGrid" class="grid-shadow"></div>
                    </div>
                    <div class="matrix-wrapper">
                        <h3>Policy Matrix</h3>
                        <div id="hw2PolicyGrid" class="grid-shadow"></div>
                    </div>
                </div>
            </div>

            <div class="result-card highlight-card">
                <h2 class="section-title optimal-title">HW1-3: Value Iteration <span>(Optimal Policy)</span></h2>
                <div class="result-container">
                    <div class="matrix-wrapper">
                        <h3>Optimal Value V*(s)</h3>
                        <div id="hw3ValueGrid" class="grid-shadow"></div>
                    </div>
                    <div class="matrix-wrapper">
                        <h3>Optimal Policy</h3>
                        <div id="hw3PolicyGrid" class="grid-shadow"></div>
                    </div>
                </div>
            </div>

        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

🟢 2. 更新 `style.css` (全螢幕現代化樣式)

請將你的 `style.css` **完全替換**成以下代碼。這裡面包含了所有圓角、陰影、漸變色以及更精緻的網格設計。

```
/* --- 全局設定 --- */
:root {
    --bg-color: #f4f7f6;
    --text-main: #2d3748;
    --text-muted: #a0aec0;
    --primary: #4299e1;
    --primary-hover: #3182ce;
    --success: #48bb78;
    --success-hover: #38a169;
    --danger: #f56565;
    --gray-bg: #e2e8f0;
    --obstacle: #718096;
    --path-bg: #c6f6d5;
    --path-arrow: #22543d;
    --normal-arrow: #a0aec0;
    --border-color: #cbd5e0;
}

body {
    font-family: 'Poppins', sans-serif;
    background-color: var(--bg-color);
    color: var(--text-main);
    margin: 0;
    padding: 40px 20px;
    display: flex;
    justify-content: center;
}

/* --- 主容器 --- */
.app-container {
    max-width: 1000px;
    width: 100%;
```

```
    display: flex;
    flex-direction: column;
    align-items: center;
}

header {
    text-align: center;
    margin-bottom: 30px;
}

h1 {
    margin: 0;
    font-size: 2.5rem;
    color: #1a202c;
}

.badge {
    background: var(--primary);
    color: white;
    font-size: 1rem;
    padding: 4px 10px;
    border-radius: 12px;
    vertical-align: middle;
}

.subtitle {
    margin: 5px 0 0 0;
    color: var(--text-muted);
    font-weight: 300;
}

/* --- 卡片式設計 --- */
.card, .result-card {
    background: white;
    padding: 25px 40px;
    border-radius: 16px;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.05);
    margin-bottom: 25px;
    width: 100%;
    box-sizing: border-box;
    text-align: center;
}

.highlight-card {
    border-top: 5px solid var(--success);
}

/* --- 控制面板 --- */
.controls label {
    font-weight: 600;
    margin-right: 10px;
}

.controls input {
    padding: 8px 12px;
    font-size: 16px;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    width: 60px;
    outline: none;
    font-family: inherit;
}

.controls input:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(66, 153, 225, 0.2);
}

/* --- 按鈕樣式 --- */
button {
    padding: 10px 24px;
    font-size: 16px;
    font-family: inherit;
    font-weight: 600;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    transition: all 0.2s ease;
    margin-left: 10px;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-hover); transform: translateY(-2px); }

.btn-secondary { background: var(--gray-bg); color: var(--text-main); }
.btn-secondary:hover { background: var(--border-color); transform: translateY(-2px); }

.btn-success { background: var(--success); color: white; }
.btn-success:hover { background: var(--success-hover); transform: translateY(-2px); box-shadow: 0 4px 12px rgba(72, 187, 120, 0.3); }

.action-buttons {
```

```
    display: flex;
    justify-content: center;
    gap: 15px;
    margin-top: 25px;
}

/* --- 步驟提示 --- */
#instruction-steps {
    margin: 20px 0;
    text-align: left;
    display: inline-block;
    background: #f8fafc;
    padding: 15px 30px;
    border-radius: 12px;
    border: 1px dashed var(--border-color);
}

#instruction-steps p {
    color: var(--text-muted);
    margin: 8px 0;
    font-size: 15px;
    transition: all 0.3s ease;
}

#instruction-steps p.active-step {
    color: var(--primary);
    font-weight: 700;
    transform: translateX(5px);
}

#step4 { color: var(--success) !important; font-weight: 700; }

/* --- 網格容器 --- */
.grid-wrapper {
    display: flex;
    justify-content: center;
}

#gridContainer, .grid-shadow {
    display: inline-flex;
    flex-direction: column;
    border: 2px solid #2d3748; /* 外框加粗 */
    border-radius: 8px;
    overflow: hidden; /* 讓圓角生效 */
    background-color: white;
}

.grid-shadow {
    box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
    border: 1px solid var(--border-color);
}

.grid-row { display: flex; }

.grid-cell {
    width: 65px;
    height: 65px;
    border: 1px solid var(--gray-bg);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    box-sizing: border-box;
    font-size: 16px;
    user-select: none;
    transition: background-color 0.2s ease;
    font-weight: 500;
}

.grid-cell:hover { background-color: #f7fafc; }

/* --- 單元格狀態顏色 (Material Palette) --- */
.cell-start { background-color: var(--success) !important; color: white; border-color: var(--success); }
.cell-end { background-color: var(--danger) !important; color: white; border-color: var(--danger); }
.cell-obstacle { background-color: var(--obstacle) !important; color: #e2e8f0; border-color: var(--obstacle); }
.cell-path { background-color: var(--path-bg) !important; border-color: #9ae6b4; }

/* --- 箭頭與文字 --- */
.arrow { font-size: 28px; font-weight: bold; line-height: 1; }
.arrow-path { color: var(--path-arrow); }
.arrow-normal { color: var(--normal-arrow); }

.small-text {
    font-size: 11px;
    font-weight: 700;
    letter-spacing: 0.5px;
    margin-bottom: -2px;
}

/* --- 結果區塊排版 --- */
```

```
.section-title {
    font-size: 1.5rem;
    margin-bottom: 20px;
    color: #4a5568;
}

.section-title span {
    font-size: 1rem;
    font-weight: 400;
    color: var(--text-muted);
}

.optimal-title { color: var(--success); }

.result-container {
    display: flex;
    justify-content: center;
    gap: 40px;
    flex-wrap: wrap;
}

.matrix-wrapper h3 {
    margin-bottom: 15px;
    font-size: 1.1rem;
    color: var(--text-main);
}
```

✨ 這次美化帶來的改變：

1. **懸停動態 (Hover Animations)**：現在滑過按鈕時，按鈕會微微向上浮起並產生陰影；滑過空白網格時，格子也會有淡淡的反白。
2. **指示區塊優化**：Step 123 被裝進了一個帶有虛線邊框的可愛面板裡，並且當前步驟會變成顯眼的藍色，還會稍微向右縮排，動態感十足。
3. **區分結果重點**：HW1-3 (Optimal Policy) 的結果卡片上方加了一條綠色的高亮粗線，讓教授一眼就能看出哪邊才是「最佳政策」的最終產出。

打開瀏覽器看看，這個介面是不是瞬間有了 2025 年現代網頁服務的水準了！

You

你給的 .html 和 .css 都是直接整個檔案覆蓋貼上嗎

Gemini

沒錯！就是直接「全部覆蓋」貼上。

因為這次的美化工程，我幫你把整個網頁的「骨架（HTML）」和「衣服（CSS）」都重新設計過了：

- 在 `index.html` 裡，我新增了很多用來排版的外層容器（像是 `<div class="app-container">` 和 `<div class="card">`）。
- 在 `style.css` 裡，我把原本的樣式全部清空，換成了全新的 Material Design 色系和排版邏輯。

所以，如果你把新舊程式碼混在一起，畫面絕對會跑版！

📝 具體操作步驟：

1. 打開你的 **`index.html`** 👉 按 `Ctrl + A` (全選) 👉 按 `Delete` (清空) 👉 貼上我剛剛給的 HTML 程式碼 👉 存檔。
2. 打開你的 **`style.css`** 👉 按 `Ctrl + A` (全選) 👉 按 `Delete` (清空) 👉 貼上我剛剛給的 CSS 程式碼 👉 存檔。
3. **`script.js` 和 `api/index.py` (或是 `app.py`) 完全不需要動！** 保留上一回合我們寫好「防呆機制」和「路徑高亮」的版本就好。

全部覆蓋完並存檔後，重新整理網頁，你就會看到煥然一新的現代化介面了！趕快試試看，如果有哪個按鈕的顏色或排版你想微調，我們隨時可以改！

You

最後我會貼上每一個檔案的內容，幫我整理、去除多餘的部分、增加註記

Gemini

這是一個非常好的收尾習慣！把程式碼整理乾淨、加上詳細的註解，不僅能讓教授或助教在看程式碼時一目了然（這對作業中「程式碼結構與可讀性」的評分非常有幫助），未來你自己回頭看這份專案時，也能馬上回想起演算法的邏輯。

當你貼上檔案內容後，我會幫你進行以下「程式碼大掃除與精緻化」：

1. **統一排版**：確保縮排一致，刪除不必要的空白行或廢棄的舊程式碼（像是 `console.log` 等測試用代碼）。
2. **結構化註解**：在關鍵函數（如 BFS 防呆機制、Bellman Equation 計算、路徑追蹤）上方加上清楚的中文註解，說明該段程式碼的目的與運作邏輯。
3. **優化變數命名**：檢查是否有命名不夠直覺的地方，讓變數自己就能「說話」。

請把你的 `index.html`、`style.css`、`script.js` 以及後端的 Python 檔（`app.py` 或 `api/index.py`）依序貼上來吧！需要我先幫你建立一個程式碼區塊的框架，讓你方便把檔案內容一個個填進來嗎？

You

<!DOCTYPE html> <html lang="zh-TW"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>RL Grid World: Value Iteration</title> <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600;700&display=swap" rel="stylesheet"> <link rel="stylesheet" href="style.css"> </head> <body>

 <header> <h1>Grid World RL</h1> Policy Evaluation & Value Iteration </header> <label for="gridSize">Grid Size (5-9):</label> <input type="number" id="gridSize" min="5" max="9" value="5"> <button class="btn-primary" onclick="generateGrid()">Generate</button>

 <h2 id="gridTitle" style="display: none;"></h2> Step 1: Click to set START (Green) Step 2: Click to set END (Red) Step 3: Click to set Obstacles (Grey) ✨ Setup complete! Ready to calculate.

  <button id="resetBtn" class="btn-secondary" onclick="resetGrid()" style="display: none;">Reset Grid</button> <button id="calculateBtn" class="btn-success" onclick="submitToFlask()" style="display: none;">Calculate Policy</button>

  <h2 class="section-title">HW1-2: Policy Evaluation (Random Policy)</h2>   <h3>Value Function V(s)</h3>   <h3>Policy Matrix</h3>

 <h2 class="section-title optimal-title">HW1-3: Value Iteration (Optimal Policy)</h2>   <h3>Optimal Value V*(s)</h3>   <h3>Optimal Policy</h3>

<script src="script.js"></script> </body> </html>

Gemini

這份 `index.html` 的結構已經非常完整且現代化了！

我幫你把程式碼裡因為複製貼上產生的「不正常空白字元」全部清理乾淨，統一了縮排格式，並且在每一個重要的區塊上方加上了**詳細的 HTML 註解**。這樣以後不管是你還是助教在看這份檔案時，都能一眼看出每個 `<div>` 負責的畫面區塊是什麼。

請直接複製以下程式碼覆蓋你的 `index.html`：

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>RL Grid World: Value Iteration</title>

    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="app-container">

        <header>
            <h1>Grid World <span class="badge">RL</span></h1>
            <p class="subtitle">Policy Evaluation & Value Iteration</p>
        </header>

        <div class="controls card">
            <label for="gridSize">Grid Size (5-9):</label>
            <input type="number" id="gridSize" min="5" max="9" value="5">
            <button class="btn-primary" onclick="generateGrid()">Generate</button>
        </div>

        <div class="setup-section card">
            <h2 id="gridTitle" style="display: none;"></h2>

            <div id="instruction-steps">
                <p id="step1">Step 1: Click to set <strong>START</strong> (Green)</p>
                <p id="step2">Step 2: Click to set <strong>END</strong> (Red)</p>
                <p id="step3">Step 3: Click to set <strong>Obstacles</strong> (Grey)</p>
                <p id="step4" style="display:none;">✨ Setup complete! Ready to calculate.</p>
            </div>

            <div class="grid-wrapper">
                <div id="gridContainer"></div>
            </div>

            <div class="action-buttons">
                <button id="resetBtn" class="btn-secondary" onclick="resetGrid()" style="display: none;">Reset Grid</button>
                <button id="calculateBtn" class="btn-success" onclick="submitToFlask()" style="display: none;">Calculate Policy</button>
            </div>
        </div>

        <div id="results" style="display: none;">

            <div class="result-card">
                <h2 class="section-title">HW1-2: Policy Evaluation <span>(Random Policy)</span></h2>
                <div class="result-container">
                    <div class="matrix-wrapper">
                        <h3>Value Function V(s)</h3>
                        <div id="hw2ValueGrid" class="grid-shadow"></div>
                    </div>
                    <div class="matrix-wrapper">
                        <h3>Policy Matrix</h3>
```

```
                    <div id="hw2PolicyGrid" class="grid-shadow"></div>
                </div>
            </div>
        </div>

        <div class="result-card highlight-card">
            <h2 class="section-title optimal-title">HW1-3: Value Iteration <span>(Optimal Policy)</span></h2>
            <div class="result-container">
                <div class="matrix-wrapper">
                    <h3>Optimal Value V*(s)</h3>
                    <div id="hw3ValueGrid" class="grid-shadow"></div>
                </div>
                <div class="matrix-wrapper">
                    <h3>Optimal Policy</h3>
                    <div id="hw3PolicyGrid" class="grid-shadow"></div>
                </div>
            </div>
        </div>

    </div>
</div>

<script src="script.js"></script>
</body>
</html>
```

這份 HTML 已經整理得乾乾淨淨了。接下來請貼上你的 **`style.css`**，我來幫你把樣式表的註解補齊、排版優化！

You

/* --- 全局設定 --- */ :root { --bg-color: #f4f7f6; --text-main: #2d3748; --text-muted: #a0aec0; --primary: #4299e1; --primary-hover: #3182ce; --success: #48bb78; --success-hover: #38a169; --danger: #f56565; --gray-bg: #e2e8f0; --obstacle: #718096; --path-bg: #c6f6d5; --path-arrow: #22543d; --normal-arrow: #a0aec0; --border-color: #cbd5e0; }

body { font-family: 'Poppins', sans-serif; background-color: var(--bg-color); color: var(--text-main); margin: 0; padding: 40px 20px; display: flex; justify-content: center; }

/* --- 主容器 --- */ .app-container { max-width: 1000px; width: 100%; display: flex; flex-direction: column; align-items: center; }

header { text-align: center; margin-bottom: 30px; }

h1 { margin: 0; font-size: 2.5rem; color: #1a202c; }

.badge { background: var(--primary); color: white; font-size: 1rem; padding: 4px 10px; border-radius: 12px; vertical-align: middle; }

.subtitle { margin: 5px 0 0 0; color: var(--text-muted); font-weight: 300; }

/* --- 卡片式設計 --- */ .card, .result-card { background: white; padding: 25px 40px; border-radius: 16px; box-shadow: 0 4px 20px rgba(0, 0, 0, 0.05); margin-bottom: 25px; width: 100%; box-sizing: border-box; text-align: center; }

.highlight-card { border-top: 5px solid var(--success); }

/* --- 控制面板 --- */ .controls label { font-weight: 600; margin-right: 10px; }

.controls input { padding: 8px 12px; font-size: 16px; border: 1px solid var(--border-color); border-radius: 8px; width: 60px; outline: none; font-family: inherit; }

.controls input:focus { border-color: var(--primary); box-shadow: 0 0 0 3px rgba(66, 153, 225, 0.2); }

/* --- 按鈕樣式 --- */ button { padding: 10px 24px; font-size: 16px; font-family: inherit; font-weight: 600; border: none; border-radius: 8px; cursor: pointer; transition: all 0.2s ease; margin-left: 10px; }

.btn-primary { background: var(--primary); color: white; } .btn-primary:hover { background: var(--primary-hover); transform: translateY(-2px); }

.btn-secondary { background: var(--gray-bg); color: var(--text-main); } .btn-secondary:hover { background: var(--border-color); transform: translateY(-2px); }

.btn-success { background: var(--success); color: white; } .btn-success:hover { background: var(--success-hover); transform: translateY(-2px); box-shadow: 0 4px 12px rgba(72, 187, 120, 0.3); }

.action-buttons { display: flex; justify-content: center; gap: 15px; margin-top: 25px; }

/* --- 步驟提示 --- */ #instruction-steps { margin: 20px 0; text-align: left; display: inline-block; background: #f8fafc; padding: 15px 30px; border-radius: 12px; border: 1px dashed var(--border-color); }

#instruction-steps p { color: var(--text-muted); margin: 8px 0; font-size: 15px; transition: all 0.3s ease; }

#instruction-steps p.active-step { color: var(--primary); font-weight: 700; transform: translateX(5px); }

#step4 { color: var(--success) !important; font-weight: 700; }

/* --- 網格容器 --- */ .grid-wrapper { display: flex; justify-content: center; }

#gridContainer, .grid-shadow { display: inline-flex; flex-direction: column; border: 2px solid #2d3748; /* 外框加粗 / *border-radius: 8px; overflow: hidden; /* 讓圓角生效 */* background-color: white; }

.grid-shadow { box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1); border: 1px solid var(--border-color); }

.grid-row { display: flex; }

.grid-cell { width: 65px; height: 65px; border: 1px solid var(--gray-bg); display: flex; flex-direction: column; align-items: center; justify-content: center; cursor: pointer; box-sizing: border-box; font-size: 16px; user-select: none; transition: background-color 0.2s ease; font-weight: 500; }

.grid-cell:hover { background-color: #f7fafc; }

/* --- 單元格狀態顏色 (Material Palette) --- */ .cell-start { background-color: var(--success) !important; color: white; border-color: var(--success); } .cell-end { background-color: var(--danger) !important; color: white; border-color: var(--danger); } .cell-obstacle { background-color: var(--obstacle) !important; color: #e2e8f0; border-color: var(--obstacle); } .cell-path { background-color: var(--path-bg) !important; border-color: #9ae6b4; }

/* --- 箭頭與文字 --- */ .arrow { font-size: 28px; font-weight: bold; line-height: 1; } .arrow-path { color: var(--path-arrow); } .arrow-normal { color: var(--normal-arrow); }

.small-text { font-size: 11px; font-weight: 700; letter-spacing: 0.5px; margin-bottom: -2px; }

/* --- 結果區塊排版 --- */ .section-title { font-size: 1.5rem; margin-bottom: 20px; color: #4a5568; }

.section-title span { font-size: 1rem; font-weight: 400; color: var(--text-muted); }

.optimal-title { color: var(--success); }

.result-container { display: flex; justify-content: center; gap: 40px; flex-wrap: wrap; }

.matrix-wrapper h3 { margin-bottom: 15px; font-size: 1.1rem; color: var(--text-main); }

Gemini

這份 CSS 寫得非常有條理，特別是使用了 CSS 變數（`:root`）來管理顏色，這在現代網頁開發中是非常好的習慣。

我幫你把全形的空白字元清乾淨，統一了縮排，並且加上了**更具體、專業的註解**。這些註解可以幫助你在向教授或助教展示時，清楚說明你的排版巧思（例如 Flexbox 的使用、Hover 轉場動畫等）。

請直接複製以下程式碼覆蓋你的 **`style.css`**：

```
/* ========================================
   全局變數設定 (CSS Variables)
   - 使用 Material Design 色系，方便未來快速抽換主題色
======================================== */
:root {
    --bg-color: #f4f7f6;        /* 網頁背景色 (淺灰白) */
    --text-main: #2d3748;       /* 主要文字顏色 (深灰) */
    --text-muted: #a0aec0;      /* 次要/提示文字顏色 (淺灰) */

    --primary: #4299e1;         /* 主題色 (藍色) */
    --primary-hover: #3182ce;

    --success: #48bb78;         /* 成功/起點/最佳政策顏色 (綠色) */
    --success-hover: #38a169;

    --danger: #f56565;          /* 終點顏色 (紅色) */

    --obstacle: #718096;        /* 障礙物顏色 (鐵灰) */
    --gray-bg: #e2e8f0;         /* 網格預設邊框色 */

    --path-bg: #c6f6d5;         /* 最佳路徑高亮背景色 (淺綠) */
    --path-arrow: #22543d;      /* 最佳路徑上的箭頭顏色 (深綠) */
    --normal-arrow: #a0aec0;    /* 非最佳路徑的箭頭顏色 (淺灰) */
    --border-color: #cbd5e0;    /* 卡片與輸入框邊框色 */
}

/* ========================================
   基礎排版設定
======================================== */
body {
    font-family: 'Poppins', sans-serif;
    background-color: var(--bg-color);
    color: var(--text-main);
    margin: 0;
    padding: 40px 20px;
    /* 使用 Flexbox 將主內容水平置中 */
    display: flex;
    justify-content: center;
}

/* 主容器：限制最大寬度，確保在大螢幕上不會過度拉伸 */
.app-container {
    max-width: 1000px;
    width: 100%;
    display: flex;
    flex-direction: column;
    align-items: center;
}

/* ========================================
   標題與卡片式設計 (Card UI)
======================================== */
header {
    text-align: center;
    margin-bottom: 30px;
}

h1 {
    margin: 0;
    font-size: 2.5rem;
    color: #1a202c;
```

```
}

/* 標題旁的小標籤 (RL) */
.badge {
    background: var(--primary);
    color: white;
    font-size: 1rem;
    padding: 4px 10px;
    border-radius: 12px;
    vertical-align: middle;
}

.subtitle {
    margin: 5px 0 0 0;
    color: var(--text-muted);
    font-weight: 300;
}

/* 共用卡片樣式：帶有柔和陰影與圓角 */
.card, .result-card {
    background: white;
    padding: 25px 40px;
    border-radius: 16px;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.05);
    margin-bottom: 25px;
    width: 100%;
    box-sizing: border-box;
    text-align: center;
}

/* 最佳政策結果卡的頂部綠色高亮線 */
.highlight-card {
    border-top: 5px solid var(--success);
}

/* ========================================
   控制面板與表單輸入
======================================== */
.controls label {
    font-weight: 600;
    margin-right: 10px;
}

.controls input {
    padding: 8px 12px;
    font-size: 16px;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    width: 60px;
    outline: none;
    font-family: inherit;
}

/* 輸入框聚焦時的藍色光暈 */
.controls input:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(66, 153, 225, 0.2);
}

/* ========================================
   按鈕與互動特效 (Buttons)
======================================== */
button {
    padding: 10px 24px;
    font-size: 16px;
    font-family: inherit;
    font-weight: 600;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    transition: all 0.2s ease; /* 按下與懸停的平滑轉場 */
    margin-left: 10px;
}

/* 利用 transform 做出滑過按鈕微微上浮的效果 */
.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-hover); transform: translateY(-2px); }

.btn-secondary { background: var(--gray-bg); color: var(--text-main); }
.btn-secondary:hover { background: var(--border-color); transform: translateY(-2px); }

.btn-success { background: var(--success); color: white; }
.btn-success:hover {
    background: var(--success-hover);
    transform: translateY(-2px);
    box-shadow: 0 4px 12px rgba(72, 187, 120, 0.3);
}

.action-buttons {
    display: flex;
    justify-content: center;
```

```
    gap: 15px; /* 按鈕之間的間距 */
    margin-top: 25px;
}

/* ========================================
   步驟提示區塊 (Step Instructions)
======================================== */
#instruction-steps {
    margin: 20px 0;
    text-align: left;
    display: inline-block;
    background: #f8fafc;
    padding: 15px 30px;
    border-radius: 12px;
    border: 1px dashed var(--border-color);
}

#instruction-steps p {
    color: var(--text-muted);
    margin: 8px 0;
    font-size: 15px;
    transition: all 0.3s ease;
}

/* 正在進行中的步驟高亮特效 */
#instruction-steps p.active-step {
    color: var(--primary);
    font-weight: 700;
    transform: translateX(5px); /* 微微向右縮排突出顯示 */
}

#step4 { color: var(--success) !important; font-weight: 700; }

/* ========================================
   互動網格系統 (Grid System)
======================================== */
.grid-wrapper {
    display: flex;
    justify-content: center;
}

/* 網格主容器 */
#gridContainer, .grid-shadow {
    display: inline-flex;
    flex-direction: column;
    border: 2px solid #2d3748;
    border-radius: 8px;
    overflow: hidden; /* 防止內部格子超出圓角 */
    background-color: white;
}

/* 結果矩陣特有的陰影效果 */
.grid-shadow {
    box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
    border: 1px solid var(--border-color);
}

.grid-row { display: flex; }

/* 單一網格樣式 */
.grid-cell {
    width: 65px;
    height: 65px;
    border: 1px solid var(--gray-bg);
    display: flex;
    flex-direction: column; /* 讓文字和箭頭可以上下排列 */
    align-items: center;
    justify-content: center;
    cursor: pointer;
    box-sizing: border-box;
    font-size: 16px;
    user-select: none;
    transition: background-color 0.2s ease;
    font-weight: 500;
}

/* 滑鼠懸停格子時的微反白效果 */
.grid-cell:hover { background-color: #f7fafc; }

/* 狀態顏色設定
   (使用 !important 確保覆蓋 :hover 和預設顏色)
*/
.cell-start { background-color: var(--success) !important; color: white; border-color: var(--success); }
.cell-end { background-color: var(--danger) !important; color: white; border-color: var(--danger); }
.cell-obstacle { background-color: var(--obstacle) !important; color: #e2e8f0; border-color: var(--obstacle); }
.cell-path { background-color: var(--path-bg) !important; border-color: #9ae6b4; }

/* ========================================
   矩陣結果：箭頭與文字排版
======================================== */
```

```
.arrow {
    font-size: 28px;
    font-weight: bold;
    line-height: 1;
}
.arrow-path { color: var(--path-arrow); } /* 最佳路徑箭頭：深色 */
.arrow-normal { color: var(--normal-arrow); } /* 其他箭頭：淺灰 */

/* 起點/終點的輔助小字 */
.small-text {
    font-size: 11px;
    font-weight: 700;
    letter-spacing: 0.5px;
    margin-bottom: -2px;
}

/* ========================================
   結果區塊 (Results Section)
======================================== */
.section-title {
    font-size: 1.5rem;
    margin-bottom: 20px;
    color: #4a5568;
}

.section-title span {
    font-size: 1rem;
    font-weight: 400;
    color: var(--text-muted);
}

.optimal-title { color: var(--success); }

/* 並排顯示 Value 和 Policy 矩陣 */
.result-container {
    display: flex;
    justify-content: center;
    gap: 40px;
    flex-wrap: wrap; /* 螢幕不夠寬時自動換行 */
}

.matrix-wrapper h3 {
    margin-bottom: 15px;
    font-size: 1.1rem;
    color: var(--text-main);
}
```

接著，請貼上你的 **`script.js`**，我們來整理負責邏輯的心臟部分！

You

let n = 0; let clickState = 0; let obstaclesLeft = 0; let mapData = { size: 0, start: null, end: null, obstacles: [] };

window.onload = function() { generateGrid(); };

function generateGrid() { const inputVal = document.getElementById('gridSize').value; n = parseInt(inputVal); if (n < 5 || n > 9) return alert("Please enter a number between 5 and 9.");

clickState = 0; obstaclesLeft = n - 2; mapData = { size: n, start: null, end: null, obstacles: [] }; document.getElementById('gridTitle').style.display = 'block'; document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`; document.getElementById('calculateBtn').style.display = 'none'; document.getElementById('resetBtn').style.display = 'block'; document.getElementById('results').style.display = 'none'; updateInstruction();

const container = document.getElementById('gridContainer'); container.innerHTML = '';

let cellNumber = 1; for (let i = 0; i < n; i++) { const rowDiv = document.createElement('div'); rowDiv.className = 'grid-row'; for (let j = 0; j < n; j++) { const cellDiv = document.createElement('div'); cellDiv.className = 'grid-cell'; cellDiv.innerText = cellNumber++; cellDiv.onclick = (e) => handleCellClick(e.target, i, j); rowDiv.appendChild(cellDiv); } container.appendChild(rowDiv); } }

function handleCellClick(cellElement, i, j) { if (cellElement.classList.contains('cell-start') || cellElement.classList.contains('cell-end') || cellElement.classList.contains('cell-obstacle')) return;

if (clickState === 0) { cellElement.classList.add('cell-start'); mapData.start = [i, j]; clickState = 1; } else if (clickState === 1) { cellElement.classList.add('cell-end'); mapData.end = [i, j]; clickState = 2; } else if (clickState === 2) { cellElement.classList.add('cell-obstacle'); mapData.obstacles.push([i, j]); obstaclesLeft--; if (obstaclesLeft === 0) { clickState = 3; document.getElementById('calculateBtn').style.display = 'block'; } } updateInstruction(); }

function updateInstruction() { // 1. 先把所有步驟的「高亮狀態」移除 document.getElementById('step1').classList.remove('active-step'); document.getElementById('step2').classList.remove('active-step'); document.getElementById('step3').classList.remove('active-step'); document.getElementById('step4').style.display = 'none';

// 2. 根據目前的點擊狀態，點亮對應的步驟 if (clickState === 0) { document.getElementById('step1').classList.add('active-step'); document.getElementById('step3').innerText = "Step 3: Click to set Obstacles (Grey)"; } else if (clickState === 1) { document.getElementById('step2').classList.add('active-step'); } else if (clickState === 2) { document.getElementById('step3').classList.add('active-step'); document.getElementById('step3').innerText = `Step 3: Click to set Obstacles (Grey). ${obstaclesLeft} left.`; } else if (clickState === 3) { // 設定完成，顯示完成文字 document.getElementById('step4').style.display = 'block'; } }

// 新增的 Reset 函數 function resetGrid() { // 其實我們原本寫好的 generateGrid() 裡面就包含了重置所有變數和清空網格的邏輯 // 所以直接呼叫它，就能完美達到 Reset 的效果！ generateGrid(); }

async function submitToFlask() { if (!hasValidPath()) { alert("⚠️ 錯誤：起點和終點之間被障礙物完全阻擋，找不到可通行的路徑！請點擊 Reset Grid 重新設定。"); return; // 直接中斷執行，不會把資料傳給 Flask 浪費運算資源 }

try { const response = await fetch('/api/calculate', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(mapData) });

if (!response.ok) { throw new Error(HTTP error! status: ${response.status}); } const data = await response.json(); document.getElementById('results').style.display = 'block'; // 畫出結果，HW3 額外傳入 optimal_path renderResultGrid('hw2ValueGrid', data.hw2_value, 'value'); renderResultGrid('hw2PolicyGrid', data.hw2_policy, 'policy'); renderResultGrid('hw3ValueGrid', data.hw3_value, 'value'); renderResultGrid('hw3PolicyGrid', data.hw3_policy, 'policy', data.hw3_path); } catch (error) { console.error("Error:", error); alert("無法連接到 Flask，請確保 app.py 正在執行中 !"); } }

// 加入 optimalPath 參數，用來高亮最佳路徑 function renderResultGrid(containerId, matrixData, type, optimalPath = null) { const container = document.getElementById(containerId); container.innerHTML = ''; // 使用比較粗的箭頭符號 const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

for (let i = 0; i < n; i++) { const rowDiv = document.createElement('div'); rowDiv.className = 'grid-row'; for (let j = 0; j < n; j++) { const cellDiv = document.createElement('div'); cellDiv.className = 'grid-cell'; const isStart = (mapData.start[0] === i && mapData.start[1] === j); const isEnd = (mapData.end[0] === i && mapData.end[1] === j); const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j); const isPath = optimalPath && optimalPath.some(p => p[0] === i && p[1] === j); if (isObstacle) { cellDiv.classList.add('cell-obstacle'); } else { // 如果是在最佳路徑上，加上高亮 class if (isPath) { cellDiv.classList.add('cell-path'); }

if (type === 'value') { if (isEnd) cellDiv.innerText = 'END'; else cellDiv.innerText = matrixData[i][j]; } else if (type === 'policy') { let content = ''; // 根據圖片，START 跟 END 要有小字 if (isStart) content += ' START '; if (!isEnd) { const actionList = matrixData[i][j]; // 沒有 action 時 (例如被障礙物包圍)，不顯示箭頭 if (actionList.length > 0) { const arrowStr = actionList.map(a => arrows[a]).join(''); const arrowClass = isPath ? 'arrow-path' : 'arrow-normal'; content += <div class="arrow ${arrowClass}">${arrowStr} ; } } else { content += ' END '; } cellDiv.innerHTML = content; } } rowDiv.appendChild(cellDiv); } container.appendChild(rowDiv); } }

// --- 新增：使用 BFS (廣度優先搜尋) 檢查是否有路徑 --- function hasValidPath() { const size = mapData.size; const start = mapData.start; const end = mapData.end; const obstacles = mapData.obstacles;

// 判斷是否為障礙物的輔助函數 const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c); // 建立一個紀錄是否走過的矩陣 let visited = Array.from({ length: size }, () => Array(size).fill(false)); // 將起點放入佇列 let queue = [start]; visited[start[0]][start[1]] = true;

// 上下左右四個方向 const dirs = [[-1, 0], [1, 0], [0, -1], [0, 1]];

while (queue.length > 0) { let current = queue.shift(); let r = current[0]; let c = current[1];

// 如果走到終點，代表有路徑！ if (r === end[0] && c === end[1]) { return true; }

// 探索四個方向 for (let dir of dirs) { let nextR = r + dir[0]; let nextC = c + dir[1];

// 檢查是否超出邊界 if (nextR >= 0 && nextR < size && nextC >= 0 && nextC < size) { // 如果不是障礙物，且還沒走過 if (!isObstacle(nextR, nextC) && !visited[nextR][nextC]) { visited[nextR][nextC] = true; queue.push([nextR, nextC]); } } } } // 佇列都清空了還是沒找到終點，代表被死路包圍 return false; }

Gemini

這份 `script.js` 寫得非常扎實！邏輯清晰，特別是你加上去的 BFS 防呆機制，讓整個應用程式變得非常健壯。

我幫你把整份程式碼重新梳理了一遍。除了去除不必要的隱藏全形空白，我還針對幾個主要的功能區塊（網格生成、互動邏輯、API 請求、畫面渲染、防呆尋路）加上了**詳細的區塊註解與說明**。這樣一來，整份檔案就如同教科書範例般清晰易懂。

請直接複製以下程式碼，覆蓋你的 **`script.js`**：

```
/* ========================================
   全局狀態管理 (Global State)
======================================== */
let n = 0;                  // 網格維度 (n x n)
let clickState = 0;         // 狀態機：0(等起點), 1(等終點), 2(等障礙物), 3(設定完成)
let obstaclesLeft = 0;      // 剩餘可設定的障礙物數量

// 儲存目前地圖配置，準備傳遞給後端 Flask API
let mapData = {
    size: 0,
    start: null,
    end: null,
    obstacles: []
};

// 網頁載入完成後，自動生成預設網格
window.onload = function() {
    generateGrid();
};

/* ========================================
   網格生成與初始化 (Grid Initialization)
======================================== */
function generateGrid() {
    const inputVal = document.getElementById('gridSize').value;
    n = parseInt(inputVal);

    // 驗證輸入範圍
    if (n < 5 || n > 9) return alert("Please enter a number between 5 and 9.");

    // 重置所有狀態與地圖資料
    clickState = 0;
    obstaclesLeft = n - 2;
    mapData = { size: n, start: null, end: null, obstacles: [] };

    // 更新 UI 顯示狀態
```

```
    document.getElementById('gridTitle').style.display = 'block';
    document.getElementById('gridTitle').innerText = `${n} x ${n} Square:`;
    document.getElementById('calculateBtn').style.display = 'none';
    document.getElementById('resetBtn').style.display = 'block';
    document.getElementById('results').style.display = 'none';
    updateInstruction();

    // 清空並重新生成網格 DOM 元素
    const container = document.getElementById('gridContainer');
    container.innerHTML = '';

    let cellNumber = 1;
    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';
            cellDiv.innerText = cellNumber++;
            // 綁定點擊事件，傳入網格座標 (i, j)
            cellDiv.onclick = (e) => handleCellClick(e.target, i, j);
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}

// 快速重置網格
function resetGrid() {
    generateGrid();
}

/* ========================================
   使用者點擊互動邏輯 (User Interaction)
======================================== */
function handleCellClick(cellElement, i, j) {
    // 避免重複點擊已經設定過的格子
    if (cellElement.classList.contains('cell-start') ||
        cellElement.classList.contains('cell-end') ||
        cellElement.classList.contains('cell-obstacle')) return;

    // 依據狀態機進行不同動作
    if (clickState === 0) {
        // 設定起點
        cellElement.classList.add('cell-start');
        mapData.start = [i, j];
        clickState = 1;
    } else if (clickState === 1) {
        // 設定終點
        cellElement.classList.add('cell-end');
        mapData.end = [i, j];
        clickState = 2;
    } else if (clickState === 2) {
        // 設定障礙物
        cellElement.classList.add('cell-obstacle');
        mapData.obstacles.push([i, j]);
        obstaclesLeft--;

        // 障礙物扣完即完成設定
        if (obstaclesLeft === 0) {
            clickState = 3;
            document.getElementById('calculateBtn').style.display = 'block';
        }
    }
    updateInstruction();
}

// 動態更新上方步驟提示的樣式
function updateInstruction() {
    // 清除所有高亮狀態
    document.getElementById('step1').classList.remove('active-step');
    document.getElementById('step2').classList.remove('active-step');
    document.getElementById('step3').classList.remove('active-step');
    document.getElementById('step4').style.display = 'none';

    // 點亮當前應進行的步驟
    if (clickState === 0) {
        document.getElementById('step1').classList.add('active-step');
        document.getElementById('step3').innerText = "Step 3: Click to set Obstacles (Grey)";
    } else if (clickState === 1) {
        document.getElementById('step2').classList.add('active-step');
    } else if (clickState === 2) {
        document.getElementById('step3').classList.add('active-step');
        document.getElementById('step3').innerText = `Step 3: Click to set Obstacles (Grey). ${obstaclesLeft} left.`;
    } else if (clickState === 3) {
        document.getElementById('step4').style.display = 'block';
    }
}

/* ========================================
```

```
   API 請求與資料處理 (API Request)
========================================= */
async function submitToFlask() {
    // 【防呆機制】檢查是否有路可走
    if (!hasValidPath()) {
        alert("⚠️ 錯誤：起點和終點之間被障礙物完全阻擋，找不到可通行的路徑！請點擊 Reset Grid 重新設定。");
        return;
    }

    try {
        // 將地圖配置發送給 Flask / Vercel 後端
        const response = await fetch('/api/calculate', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(mapData)
        });

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const data = await response.json();

        // 顯示結果區塊
        document.getElementById('results').style.display = 'block';

        // 渲染四個不同的結果矩陣
        renderResultGrid('hw2ValueGrid', data.hw2_value, 'value');
        renderResultGrid('hw2PolicyGrid', data.hw2_policy, 'policy');
        renderResultGrid('hw3ValueGrid', data.hw3_value, 'value');
        // HW3 Policy 額外傳入最佳路徑 (optimal_path) 以進行高亮渲染
        renderResultGrid('hw3PolicyGrid', data.hw3_policy, 'policy', data.hw3_path);

    } catch (error) {
        console.error("Error:", error);
        alert("無法連接到後端伺服器，請確保 API 正在運行中 !");
    }
}

/* =========================================
   結果渲染函數 (Result Rendering)
========================================= */
function renderResultGrid(containerId, matrixData, type, optimalPath = null) {
    const container = document.getElementById(containerId);
    container.innerHTML = '';

    // 定義粗體箭頭符號
    const arrows = { 'up': '↑', 'down': '↓', 'left': '←', 'right': '→' };

    for (let i = 0; i < n; i++) {
        const rowDiv = document.createElement('div');
        rowDiv.className = 'grid-row';
        for (let j = 0; j < n; j++) {
            const cellDiv = document.createElement('div');
            cellDiv.className = 'grid-cell';

            // 判斷當前格子的屬性
            const isStart = (mapData.start[0] === i && mapData.start[1] === j);
            const isEnd = (mapData.end[0] === i && mapData.end[1] === j);
            const isObstacle = mapData.obstacles.some(obs => obs[0] === i && obs[1] === j);
            const isPath = optimalPath && optimalPath.some(p => p[0] === i && p[1] === j);

            if (isObstacle) {
                cellDiv.classList.add('cell-obstacle');
            } else {
                // 若屬於最佳路徑，加上綠色高亮背景
                if (isPath) {
                    cellDiv.classList.add('cell-path');
                }

                if (type === 'value') {
                    // 渲染 Value Matrix (數字)
                    if (isEnd) cellDiv.innerText = 'END';
                    else cellDiv.innerText = matrixData[i][j];
                } else if (type === 'policy') {
                    // 渲染 Policy Matrix (箭頭)
                    let content = '';

                    if (isStart) content += '<div class="small-text">START</div>';

                    if (!isEnd) {
                        const actionList = matrixData[i][j];
                        // 確保該格子有可行動方向
                        if (actionList.length > 0) {
                            const arrowStr = actionList.map(a => arrows[a]).join('');
                            // 判斷是否為最佳路徑上的箭頭，決定顏色深淺
                            const arrowClass = isPath ? 'arrow-path' : 'arrow-normal';
                            content += `<div class="arrow ${arrowClass}">${arrowStr}</div>`;
                        }
```

```
                    } else {
                        content += '<div class="small-text">END</div>';
                    }

                    cellDiv.innerHTML = content;
                }
            }
            rowDiv.appendChild(cellDiv);
        }
        container.appendChild(rowDiv);
    }
}

/* ========================================
   防呆尋路演算法 (BFS Pathfinding Validation)
   - 用於檢查起點與終點之間是否被障礙物阻擋
======================================== */
function hasValidPath() {
    const size = mapData.size;
    const start = mapData.start;
    const end = mapData.end;
    const obstacles = mapData.obstacles;

    // 輔助函數：判斷是否為障礙物
    const isObstacle = (r, c) => obstacles.some(obs => obs[0] === r && obs[1] === c);

    // 建立紀錄訪問狀態的 2D 陣列
    let visited = Array.from({ length: size }, () => Array(size).fill(false));

    // 初始化 BFS 佇列
    let queue = [start];
    visited[start[0]][start[1]] = true;

    // 定義上下左右位移
    const dirs = [[-1, 0], [1, 0], [0, -1], [0, 1]];

    while (queue.length > 0) {
        let current = queue.shift();
        let r = current[0];
        let c = current[1];

        // 成功尋得終點
        if (r === end[0] && c === end[1]) {
            return true;
        }

        // 探索鄰近節點
        for (let dir of dirs) {
            let nextR = r + dir[0];
            let nextC = c + dir[1];

            // 確保不越界
            if (nextR >= 0 && nextR < size && nextC >= 0 && nextC < size) {
                // 若為可行走路徑且未訪問過，加入佇列
                if (!isObstacle(nextR, nextC) && !visited[nextR][nextC]) {
                    visited[nextR][nextC] = true;
                    queue.push([nextR, nextC]);
                }
            }
        }
    }

    // 窮舉所有可達節點後仍未見終點，判定為無效路徑
    return false;
}
```

接下來是最後一塊拼圖：你部署在 `api/` 資料夾底下的 **`index.py`**（後端 Flask 檔案）。貼上來吧！

You

from flask import Flask, request, jsonify from flask_cors import CORS

app = Flask(**name**) CORS(app)

ACTIONS = {'up': (-1, 0), 'down': (1, 0), 'left': (0, -1), 'right': (0, 1)}

def get_next_state(r, c, action_name, grid_size, obstacles): dr, dc = ACTIONS[action_name] next_r, next_c = r + dr, c + dc if next_r < 0 or next_r >= grid_size or next_c < 0 or next_c >= grid_size or [next_r, next_c] in obstacles: return r, c return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0): policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue max_val = -float('inf') best_actions = [] for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) val = -1 + gamma * V[nr][nc] # 只要嚴格大於目前最大值才更新，遇到平手就不理它！ if val > max_val + 1e-6: max_val = val best_actions = [a] # 永遠只保留一個方向 policy[r][c] = best_actions return policy

def trace_optimal_path(policy, start, end): path = [] current = tuple(start) visited = set() while current != tuple(end): if current in visited: break visited.add(current) path.append(list(current)) r, c = current actions = policy[r][c] if not actions: break best_action = actions[0] dr, dc = ACTIONS[best_action] current = (r + dr, c + dc) path.append(list(end)) return path

def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0): V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] while True: delta = 0 new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue v_expected = 0 for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) v_expected += 0.25 * (-1 + gamma * V[nr][nc]) new_V[r][c] = v_expected delta = max(delta, abs(v_expected - V[r][c])) V = new_V if delta < theta: break policy = derive_policy(V, grid_size, end, obstacles, gamma) return V, policy

def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0): V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] while True: delta = 0 new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)] for r in range(grid_size): for c in range(grid_size): if [r, c] == end or [r, c] in obstacles: continue max_v = -float('inf') for a in ACTIONS: nr, nc = get_next_state(r, c, a, grid_size, obstacles) v = -1 + gamma * V[nr][nc] if v > max_v: max_v = v new_V[r][c] = max_v delta = max(delta, abs(max_v - V[r][c])) V = new_V if delta < theta: break policy = derive_policy(V, grid_size, end, obstacles, gamma) return V, policy

@app.route('/api/calculate', methods=['POST']) def calculate(): data = request.json size = data['size'] start = data['start'] end = data['end'] obstacles = data['obstacles'] v_hw2, p_hw2 = policy_evaluation(size, end, obstacles) v_hw3, p_hw3 = value_iteration(size, end, obstacles) optimal_path_hw3 = trace_optimal_path(p_hw3, start, end) v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2] v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3] return jsonify({ 'hw2_value': v_hw2_round, 'hw2_policy': p_hw2, 'hw3_value': v_hw3_round, 'hw3_policy': p_hw3, 'hw3_path': optimal_path_hw3 })

Gemini

這份後端的 Python 程式碼邏輯已經非常完美了，成功實作了強化學習中最核心的兩個演算法。

我幫你清除了隱藏的全形空白字元（這在 Python 裡很容易引發 IndentationError），並且為每一個演算法和輔助函數加上了**詳細的中文註解與 Docstring (文件字串)**。這樣一來，無論是教授檢視程式碼，或是你未來複習時，都能馬上理解背後的數學邏輯（如 Bellman Equation）。

請直接複製以下內容，覆蓋你的 **`api/index.py`**（如果你是在本機跑就覆蓋 `app.py`）：

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # 允許前端跨域請求 (CORS)

# 定義 Agent 可執行的四個行動與對應的座標位移 (row, col)
ACTIONS = {
    'up': (-1, 0),
    'down': (1, 0),
    'left': (0, -1),
    'right': (0, 1)
}

# ========================================
# MDP 環境動態與輔助函數 (Environment Dynamics)
# ========================================

def get_next_state(r, c, action_name, grid_size, obstacles):
    """
    計算給定狀態與行動下的下一個狀態。
    如果撞牆或撞到障礙物，則保持在原地。
    """
    dr, dc = ACTIONS[action_name]
    next_r, next_c = r + dr, c + dc

    # 邊界與障礙物判定
    if (next_r < 0 or next_r >= grid_size or
        next_c < 0 or next_c >= grid_size or
        [next_r, next_c] in obstacles):
        return r, c # 留在原地

    return next_r, next_c

def derive_policy(V, grid_size, end, obstacles, gamma=1.0):
    """
    根據收斂的價值函數 V(s)，推導出最佳政策 (Greedy Policy)。
    """
    policy = [[[] for _ in range(grid_size)] for _ in range(grid_size)]
    for r in range(grid_size):
        for c in range(grid_size):
            # 終點與障礙物不需要策略
            if [r, c] == end or [r, c] in obstacles:
                continue

            max_val = -float('inf')
            best_actions = []

            # 評估四個方向，找出期望價值最高的一步
            for a in ACTIONS:
                nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                val = -1 + gamma * V[nr][nc] # Reward 固定為 -1

                # 嚴格大於目前最大值才更新，避免產生多重箭頭 (打破平手)
                if val > max_val + 1e-6:
                    max_val = val
                    best_actions = [a]

            policy[r][c] = best_actions

    return policy

def trace_optimal_path(policy, start, end):
```

```
    """
    從起點開始，順著最佳政策(箭頭)走到終點，記錄下完整的最佳路徑。
    """
    path = []
    current = tuple(start)
    visited = set() # 用於防止死胡同造成的無限迴圈

    while current != tuple(end):
        if current in visited:
            break
        visited.add(current)
        path.append(list(current))

        r, c = current
        actions = policy[r][c]
        if not actions:
            break # 沒路可走

        # 順著最佳策略走
        best_action = actions[0]
        dr, dc = ACTIONS[best_action]
        current = (r + dr, c + dc)

    path.append(list(end))
    return path

# ========================================
# 強化學習核心演算法 (RL Algorithms)
# ========================================

def policy_evaluation(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    """
    HW1-2: 策略評估 (Policy Evaluation)
    評估「隨機策略」(每個方向機率各 25%) 下的狀態價值。
    使用 Bellman Expectation Equation 進行迭代。
    """
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue

                v_expected = 0
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    # 隨機策略: pi(a|s) = 0.25
                    v_expected += 0.25 * (-1 + gamma * V[nr][nc])

                new_V[r][c] = v_expected
                delta = max(delta, abs(v_expected - V[r][c]))

        V = new_V
        if delta < theta: # 收斂條件
            break

    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

def value_iteration(grid_size, end, obstacles, theta=1e-4, gamma=1.0):
    """
    HW1-3: 價值迭代 (Value Iteration)
    直接推導最佳政策下的狀態價值。
    使用 Bellman Optimality Equation (取 max) 進行迭代。
    """
    V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

    while True:
        delta = 0
        new_V = [[0.0 for _ in range(grid_size)] for _ in range(grid_size)]

        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue

                max_v = -float('inf')
                for a in ACTIONS:
                    nr, nc = get_next_state(r, c, a, grid_size, obstacles)
                    # 取四個動作中的最大價值 (Greedy)
                    v = -1 + gamma * V[nr][nc]
                    if v > max_v:
                        max_v = v

                new_V[r][c] = max_v
                delta = max(delta, abs(max_v - V[r][c]))
```

```
        V = new_V
        if delta < theta: # 收斂條件
            break

    policy = derive_policy(V, grid_size, end, obstacles, gamma)
    return V, policy

# ========================================
# Flask API 路由 (API Routes)
# ========================================

@app.route('/api/calculate', methods=['POST'])
def calculate():
    """接收前端的地圖資料，執行 RL 演算法並回傳結果"""
    data = request.json
    size = data['size']
    start = data['start']
    end = data['end']
    obstacles = data['obstacles']

    # 執行 HW1-2: 策略評估 (Random Policy)
    v_hw2, p_hw2 = policy_evaluation(size, end, obstacles)

    # 執行 HW1-3: 價值迭代 (Optimal Policy)
    v_hw3, p_hw3 = value_iteration(size, end, obstacles)

    # 追蹤 HW1-3 的最佳路徑
    optimal_path_hw3 = trace_optimal_path(p_hw3, start, end)

    # 將價值矩陣四捨五入到小數點後兩位，方便前端顯示
    v_hw2_round = [[round(val, 2) for val in row] for row in v_hw2]
    v_hw3_round = [[round(val, 2) for val in row] for row in v_hw3]

    # 回傳 JSON 格式結果
    return jsonify({
        'hw2_value': v_hw2_round,
        'hw2_policy': p_hw2,
        'hw3_value': v_hw3_round,
        'hw3_policy': p_hw3,
        'hw3_path': optimal_path_hw3
    })

# 若需在本地端測試，可將下方兩行取消註解
# if __name__ == '__main__':
#     app.run(debug=True, port=5000)
```

🎉 大功告成！

恭喜你！這份作業現在不僅完美達成了 HW1-1、HW1-2、HW1-3 的所有要求，還具備了：

- 🚀 **無伺服器部署能力 (Vercel Ready)**
- 🛡️ **完整的防呆機制 (BFS 無效路徑阻擋)**
- ✨ **現代化的 UI/UX 設計 (Material Design & Animations)**
- 📖 **極具可讀性的程式碼結構與註解**

你現在可以放心地把這包程式碼 push 到 GitHub 並透過 Vercel 部署了。祝你這份作業拿高分！如果之後還有其他強化學習的作業或問題，隨時回來找我討論！

You

以下是 readme.md 的內容

根據新增的部分完善它

Deep Reinforcement Learning - HW1: Grid World Policy Evaluation

👉 **點此觀看 Live Demo**

🌟 功能 (Features)

HW1-1: 網格地圖開發 (前端互動)

- **動態網格生成**：允許用戶自由輸入維度 n (範圍從 5 到 9)，動態生成 n x n 的網格地圖。
- **狀態機互動設計**：透過流暢的滑鼠點擊流程設定環境： 1. **起點 (Start)**：點擊設定為綠色單元格。 2. **終點 (End)**：點擊設定為紅色單元格。 3. **障礙物 (Obstacles)**：設定 n-2 個障礙物，點擊變為灰色單元格（保留原始數字顯示）。

HW1-2: 策略顯示與價值評估 (核心演算法)

- **策略評估 (Policy Evaluation)**：基於隨機策略（上、下、左、右機率各 25%），透過貝爾曼期望方程式 (Bellman Expectation Equation) 迭代計算出每個狀態的價值 V(s)，直到數值收斂。
- **策略矩陣 (Policy Matrix)**：根據算出的 V(s) 推導出 Greedy Policy，在畫面上顯示每個單元格的最佳行動方向（↑, ↓, ←, →），若有多個最佳方向亦會同時顯示。

- **價值矩陣 (Value Matrix)**：清楚呈現每個單元格最終收斂的預期價值（四捨五入至小數點後兩位）。

🚀 如何使用 (How to Use)

1. 開啟 Demo 網頁。
2. 在左上角的輸入框輸入 5 到 9 之間的數字，並點擊 **"Generate Square"** 生成網格。
3. 根據畫面上方的提示，依序點擊網格來設定： * 第 1 下：設定**起點** (綠色)。 * 第 2 下：設定**終點** (紅色)。 * 接下來的 n-2 下：設定**障礙物** (灰色)。
4. 設定完成後，點擊下方出現的 **"計算策略與價值 (Calculate)"** 按鈕。
5. 頁面下方會立刻生成並顯示計算完成的 **Value Matrix** (數值) 與 **Policy Matrix** (方向箭頭)。

Gemini

這份 README 的基礎已經打得很好了！結合我們後來加入的 **HW1-3 (價值迭代與路徑高亮)**、**防呆機制 (BFS)**、**現代化 UI** 以及 **Flask/Vercel 後端架構**，我幫你大幅擴充並完善了這份文件。

這份新的 README 不僅涵蓋了所有作業要求，還凸顯了你專案中的「加分亮點」，讓教授或助教在評分時能一眼看出你的用心：

```
# Deep Reinforcement Learning - HW1: Grid World Policy Evaluation & Value Iteration

本專案是一個基於強化學習 (Reinforcement Learning) 的「網格世界 (Grid World)」互動式網頁應用程式。完整實作了**策略評估 (Policy Evaluation)** 與**

👉 **[點此觀看 Live Demo](https://bibaisafish.github.io/DRL_HW1/)** *(註：若後端部署於 Vercel 等平台，請確保 API 服務正常運行)*

---

## 🌟 核心功能 (Features)

### HW1-1: 網格地圖開發 (前端互動)
* **動態網格生成**：允許用戶自由輸入維度 `n` (範圍從 5 到 9)，動態生成 `n x n` 的網格地圖。
* **狀態機互動設計**：透過流暢的滑鼠點擊流程設定環境：
  1. **起點 (Start)**：點擊設定為綠色單元格。
  2. **終點 (End)**：點擊設定為紅色單元格。
  3. **障礙物 (Obstacles)**：設定 `n-2` 個障礙物，點擊變為灰色單元格（保留原始數字顯示）。

### HW1-2: 策略顯示與價值評估 (Random Policy Baseline)
* **策略評估 (Policy Evaluation)**：基於隨機策略（上、下、左、右機率各 25%），透過貝爾曼期望方程式 (Bellman Expectation Equation) 迭代計算出每個狀
* **策略矩陣 (Policy Matrix)**：根據算出的 `V(s)` 推導出 Greedy Policy，在畫面上顯示每個單元格的最佳行動方向。
* **價值矩陣 (Value Matrix)**：清楚呈現每個單元格最終收斂的預期價值（四捨五入至小數點後兩位）。

### HW1-3: 價值迭代與最佳路徑 (Optimal Policy)
* **價值迭代 (Value Iteration)**：實作貝爾曼最佳方程式 (Bellman Optimality Equation)，精準迭代出每個狀態的最大期望回報 $V^*(s)$。
* **最佳政策推導**：根據 $V^*(s)$ 推導出絕對的最佳行動方向（演算法具備打破平手機制，確保單元格呈現單一明確的箭頭）。
* **最佳路徑高亮視覺化**：後端自動追蹤從起點至終點的最短路徑。在前端渲染時，**路徑上的單元格會以綠色高亮標示且箭頭加深，非路徑的箭頭則以灰色弱化處理*

---

## 🛡️ 進階體驗與防呆機制 (UX & Validation)

* **無效路徑阻擋 (BFS Pathfinding Validation)**：為避免起點被障礙物包圍導致演算法價值發散，前端在送出請求前，會先使用「廣度優先搜尋 (BFS)」進行輕量
* **現代化 UI/UX 設計**：導入 Material Design 色系、Card UI 卡片式排版與 Hover 轉場動畫。
* **動態步驟指示器與重置**：畫面上方具備動態的高亮步驟清單，指引使用者當前操作進度；並新增 `Reset Grid` 按鈕，隨時可一鍵清空地圖重新設定。
* **前後端分離架構**：前端 (HTML/CSS/JS) 透過 Fetch API 將地圖狀態發送至 Python Flask 後端，運算完成後同時回傳 HW1-2 與 HW1-3 的結果矩陣進行同框

---

## 🚀 如何使用 (How to Use)

1. 開啟 [Demo 網頁](https://bibaisafish.github.io/DRL_HW1/)。
2. 在左上角的輸入框輸入 `5` 到 `9` 之間的數字，並點擊 **"Generate"** 生成網格。
3. 觀察畫面上方的動態步驟提示，依序點擊網格來設定：
   * **Step 1**：點擊任意空格設定**起點** (綠色)。
   * **Step 2**：點擊任意空格設定**終點** (紅色)。
   * **Step 3**：點擊剩餘空格設定 `n-2` 個**障礙物** (灰色)。
4. 設定完成後，點擊下方出現的 **"Calculate Policy"** 按鈕。
   * *(若故意將起點封死，系統會彈出防呆警告提示重置)*。
5. 頁面下方會立刻生成兩組對比結果：
   * **上半部 (HW1-2)**：隨機政策下的價值與策略矩陣。
   * **下半部 (HW1-3)**：最佳政策下的價值、策略矩陣，以及**高亮顯示的最佳路徑**。
6. 如需重新測試，點擊 **"Reset Grid"** 即可重新設定。
```

這份更新版的 README 結構非常完整，直接突出了你這份作業「不僅做完了要求，還做得比別人更好、更穩」的特點。你可以直接覆蓋掉原本的內容！